Hands-On Machine Learning

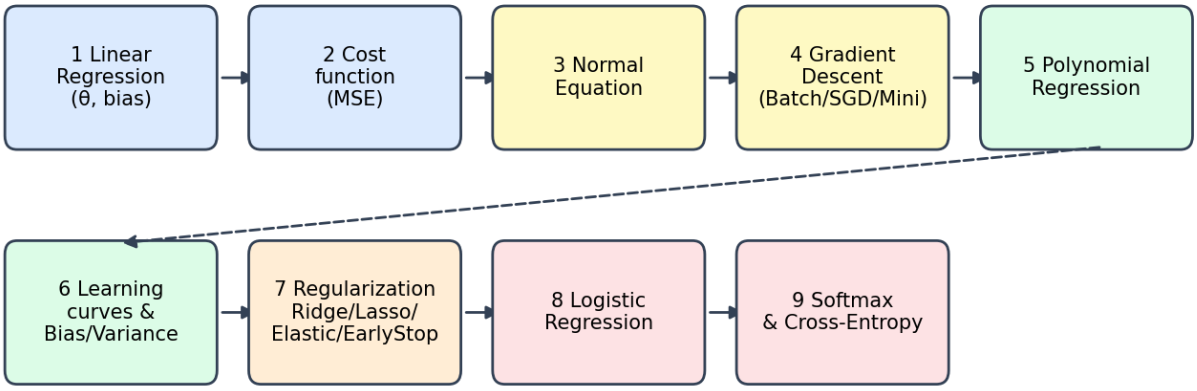
Chapter 4 — Training Models

Linear · Polynomial · Regularized · Logistic · Softmax

ملخص كشكولك + تكملة النقط الناقصة + رسومات وأكواد للمراجعة

من كشكولك | إضافة لإكمال النقص | انتبه / تصحيح

الأرقام اللي في مربعات **Output (real run)** الخضرا طلعت من تشغيل الكود فعلًا. الرسومات اللي عليها (synthetic) أو (illustrative) داتا مولّدة للتوضيح بس. رسومات Iris من داتا حقيقية.



خريطة الشابتر: كل خطوة بتحل مشكلة الخطوة اللي قبلها

1 الـ Linear Regression: الفكرة والـ Parameters

من كشكولك

Linear Regression: الفكرة بسيطة، عندي features وعايز أتوقع قيمة رقمية. ← `X -> Model -> ŷ`

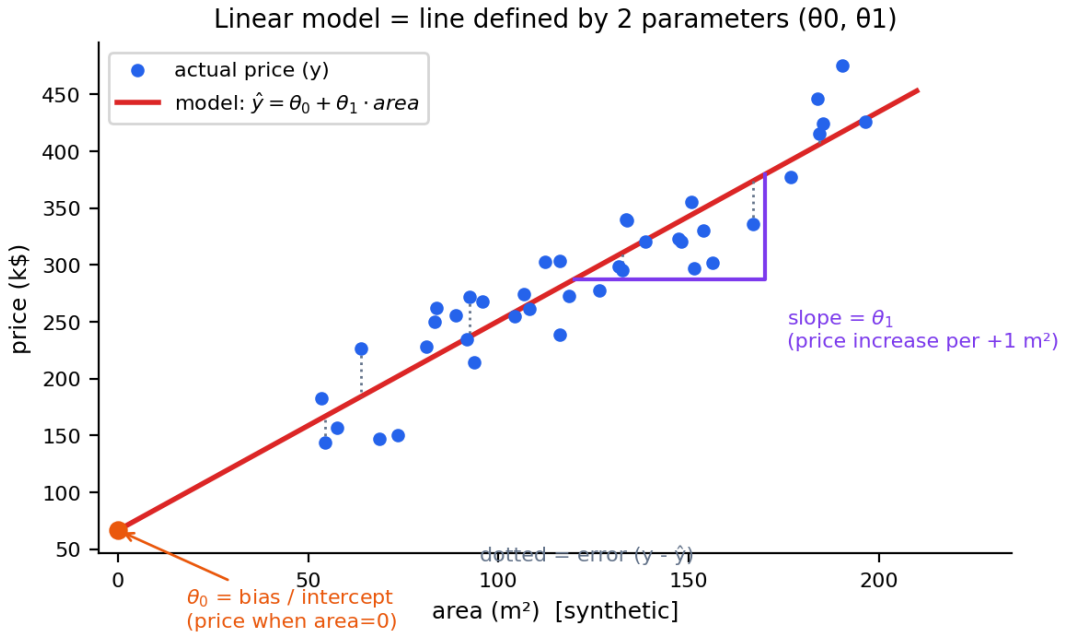
الـ linear model بيعمل التوقع عن طريق: Weighted sum of Features + Bias

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$\hat{y} = h_\theta(\mathbf{x}) = \theta \cdot \mathbf{x}$$

- `ŷ` = القيمة اللي الموديل توقعها.
- `Features = x1, x2, ..., xn`
- `weights / parameters = θ1, θ2, ..., θn`
- `Bias / Intercept = θ0` (الانحياز / الحد الثابت)

مثال: لو عندي `Price = θ0 + θ1 × area + θ2 × rooms`، الموديل بيتعلم قيم `θ0, θ1, θ2` بحيث التوقعات تكون قريبة من القيم الحقيقية.



الموديل الخطي = خط معرّف بـ parameters. θ0 هو نقطة تقاطع الخط مع محور y، وθ1 هو ميل الخط. الخطوط المنقطة = الخطأ.

من كشكولك

Parameters

Parameters: هي القيم اللي الموديل نفسه بيتعلمها من البيانات. مثلًا: `θ0 = 100, θ1 = 20, θ2 = 500`.

- الـ Linear Regression لما بيتدرب بيحاول يلاقي أفضل قيم للـ `θ` بحيث يقلل الخطأ.
- Training ← إيجاد أفضل قيم للـ Parameters.
- ودي مختلفة عن الـ Hyperparameters اللي كنا بنتعامل معاها في شابتر 2: الـ Hyperparameters **إحنا** اللي بنحددها، لكن الـ Parameters **الموديل** بيتعلمها.

إضافة

بالأرقام اللي في مثالك: لو بيت مساحته `area = 100` وعدد غرفه `rooms = 3`:

`ŷ = 100 + 20×100 + 500×3 = 3600`. كل ما الـ θ تتغير، التوقع بيتغير؛ ولذلك التدريب = البحث عن θ.

حيلة الـ Bias: بنضيف عمود كله `1` للـ features (`x0 = 1`) فنقدر نكتب المعادلة كلها كضرب vector في vector: `ŷ = θ · x`. وبالتالي الـ matrix `X` بيبقى شكلها `(m, n+1)` (عدد العينات × عدد الـ features + 1).

	Parameter	Hyperparameter
مين بيحددها؟	الموديل بيتعلمها من الداتا	إحنا بنحددها قبل التدريب
أمثلة	θ0, θ1, ... (weights)	learning rate η, عدد الـ epochs, alpha, degree
بتتغير أثناء التدريب؟	أيوه	لأ (إلا لو عملنا tuning بنفسنا)

الموديل بيتعلم الـ θ إزاي؟

من كشكولك

إحنا عارفين إن الـ linear regression بيعمل prediction باستخدام الـ `θ`، لكن مين هيختار الـ `θ` ؟ لازم نختار قيم تخلّي الموديل أفضل ما يكون على الـ training set، ولمعرفة "أفضل" لازم نستخدم Cost function.

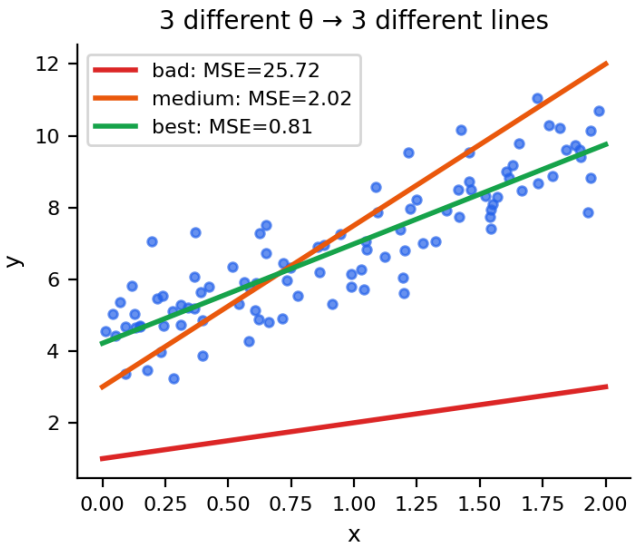
2 الـ Cost Function والفرق بين الـ Loss والـ Metric

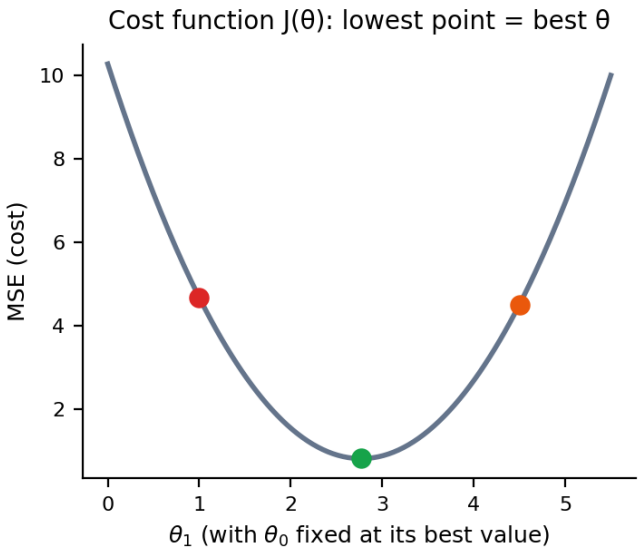
من كشكولك

Cost Function: هو أي دالة بتقيس "قد إيه الموديل سيء" بناءً على قيم `theta` معينة. مثل: `MSE` , `RMSE` (بتاخد `θ` كمدخل وترجع رقم واحد يمثل "مدى سوء الأداء").

الـ `MSE` نفسها (كمفهوم) معيار تقييم عام، لكن لما نستخدمها كدالة بتاخد `θ` وتقولك "الخطأ قد إيه مع القيم دي بالذات" فهي بقت Cost function اللي بنحاول نقلّلها.

$$\text{MSE}(\theta) = \frac{1}{m}\sum_{i=1}^{m}(\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$





شمال: 3 قيم مختلفة لـ θ = 3 خطوط مختلفة، وكل واحد له MSE. يمين: الـ cost كدالة في θ1: أقل نقطة هي أحسن θ (الأرقام من تشغيل حقيقي).

من كشكولك

نقطة مهمة: Evaluation metric ≠ Training loss بالضرورة. ممكن الـ algorithm أثناء التدريب يستخدم loss function معينة، وبعد التدريب نقيس بـ metric مختلفة.

السبب إن الـ loss قد تكون أسهل في التحسين حسابيًا، أو يكون فيها مكونات إضافية خاصة بعملية التدريب (Regularization).

← المهم إن الـ Training objective لازم يكون مرتبط بقوة بالهدف النهائي للمشروع.

إضافة

مثال من الكتاب (بنوصله في آخر الشابتر): الـ Logistic Regression بيتدرب بـ log loss لكن بنقيّمه بعد كده بـ Precision / Recall. الاتنين مختلفين، وده طبيعي.

وكمان `MSE` و `RMSE` بيوصلوا لنفس أحسن `θ` (لأن الجذر التربيعي دالة بتزيد دايمًا)، لكن `MSE` أسهل في المشتقة، فهي اللي بنستخدمها في التدريب.

انتبه / تصحيح

افتكر تصحيح شابتر 2: لو عايز `RMSE` في النسخ الحديثة من scikit-learn استخدم `root_mean_squared_error(y_true, y_pred)` بدل `mean_squared_error(..., squared=False)` اللي اتشالت.

3 الـ Normal Equation

من كشكولك

بالنسبة للـ linear regression فيه طريقة رياضية تقدر تجيب لنا أفضل Parameters **مباشرة** بدون ما نعمل Normal equation :iterations

$$\hat{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

المعنى: استخدم البيانات نفسها في معادلة رياضية فتطلع لك قيم الـ Parameters اللي تقلل الـ `MSE`.

من كشكولك

كود الكتاب (Normal equation): أول حاجة نولّد بيانات تقريبًا خطية: `y = 4 + 3X` ، لكن الكتاب أضاف Gaussian noise ولذلك لما الموديل يحاول يسترجع العلاقة مش لازم يطلع بالظبط 4 و 3.

بعدها نضيف الـ Bias column: `add_dummy_feature(X)` دي بتضيف `x0 = 1` لكل instance. وبعد كده نقدر نحسب الـ bias ضمن الـ matrix operation.

```
import numpy as np
from sklearn.preprocessing import add_dummy_feature

np.random.seed(42)
m = 100
X = 2 * np.random.rand(m, 1)
y = 4 + 3 * X + np.random.randn(m, 1)      # 4 + 3x + Gaussian noise

X_b = add_dummy_feature(X)                 # adds x0 = 1 to every instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y
print(theta_best)
```

Output (real run)

```
[[4.2151]
 [2.7701]]
```

من كشكولك

ده تنفيذ مباشر للـ Normal equation والـ `@` هو Matrix multiplication في numpy. والنتيجة طلعت `θ0 = 4.215` و `θ1 = 2.770` مش 4 و 3 بالظبط بسبب الـ noise.

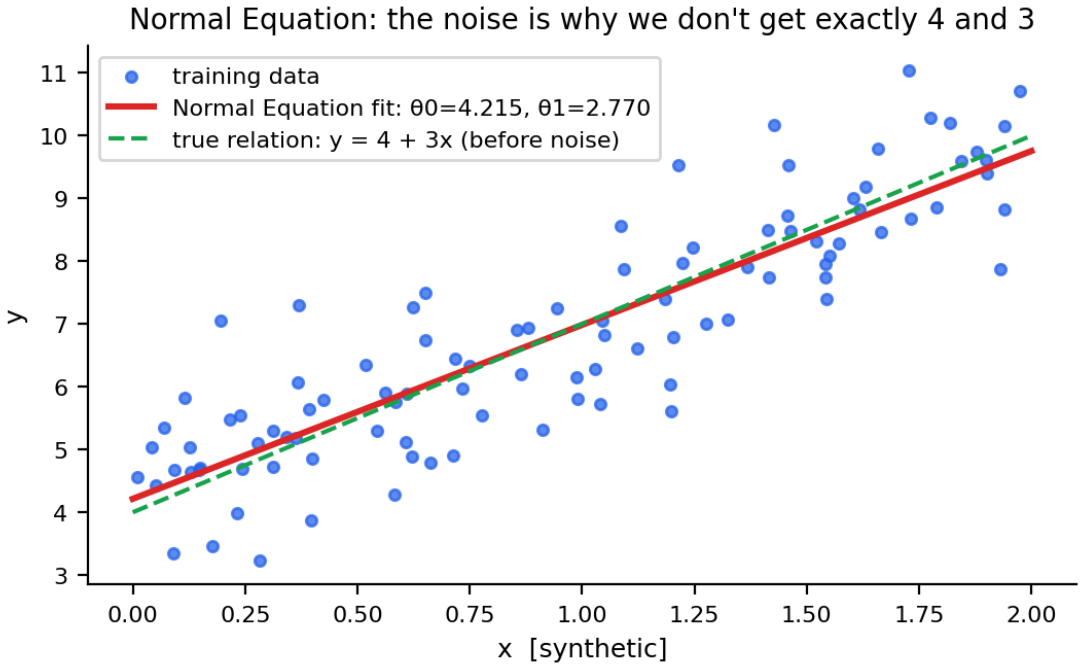
الكتاب استخدم الـ `θ` بعد ما عرفناها للـ prediction:

```
X_new = np.array([[0], [2]])
X_new_b = add_dummy_feature(X_new)        # add the bias column to the NEW data too
y_predict = X_new_b @ theta_best          # y_hat = X . theta
print(y_predict)
```

```
Output (real run)
[[4.2151]
 [9.7553]]
```

من كشكولك

يعني: خد Features جديدة، أضف bias column، اضربها في الـ Parameters (θ) وطلّع الـ prediction. وده نفس مفهوم $\hat{y} = X\theta$.



نتيجة الـ Normal Equation على الداتا دي: الخط الأحمر قريب جدًا من العلاقة الحقيقية (الأخضر) بس مش مطابق بسبب الـ noise.

من كشكولك

طب ليه بنعمل كل ده يدويًا؟ عشان تفهم اللي بيحصل تحت الـ hood، لكن في الواقع هتستخدم:

```
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()
lin_reg.fit(X, y)
print(lin_reg.intercept_, lin_reg.coef_)     # theta0 , theta1 ...
print(lin_reg.predict(X_new))
```

```
Output (real run)
[4.2151] [[2.7701]]
[[4.2151]
 [9.7553]]
```

من كشكولك

بعد التدريب: `lin_reg.intercept_` ← دي بتمثل `θ0` و `lin_reg.coef_` ← `θ1, θ2, ...`

من كشكولك

طيب الـ linear regression نفسها بتستخدم إيه؟ الكتاب بيقول إنها مبنية على `scipy.linalg.lstsq()` والمعنى Least squares. وتستخدم طريقة تعتمد على pseudoinverse بدل تنفيذ الـ normal equation بالطريقة المباشرة، لأن `X^T X` ممكن تكون مش قابلة للعكس، لكن الـ pseudoinverse بتشتغل حتى لو مش قابلة للعكس.

← Scikit-learn مش محتاج إنك تكتب Normal equation بنفسك، هي اللي بتتولى الحساب بطريقة أكتر استقرارًا (stability).

```
theta_svd = np.linalg.pinv(X_b) @ y       # same answer through the pseudoinverse
print(theta_svd)
```

```
Output (real run)
[[4.2151]
 [2.7701]]
```

من كشكولك

ليه مش بنستخدم الـ normal equation دايمًا؟ الـ normal equation محتاجة حساب `(X^T X)^-1` ، وكل ما زاد عدد الـ features `n` تصبح عملية Matrix inversion أغلى حسابيًا. والكتاب بيذكر إن التكلفة ممكن تكون `O(n^2.4)` إلى `O(n^3)` حسب التنفيذ. كذلك لو عدد الـ features كبير جدًا الطرق دي تصبح بطيئة.

إضافة

إمتى `X^T X` ما تبقاش قابلة للعكس؟ لما يكون فيه features مكررة أو مرتبطة ببعض تمامًا (مثلًا `area_m2` و `area_ft2` الاتنين مع بعض)، أو لما عدد العينات `m` أقل من عدد الـ features `n`. ساعتها الـ `inv` بتفشل أو بتدي نتيجة مش مستقرة، والـ pseudoinverse (`pinv`) هي الحل.

ونقطة حلوة: الـ Normal Equation **خطية** مع عدد العينات `m` (يعني 10 أضعاف الداتا = 10 أضعاف الوقت تقريبًا)، فمشكلتها الحقيقية في كبر `n` مش كبر `m`. وبعد التدريب التوقع نفسه سريع جدًا (خطي في `m` و `n`).

4 الـ Gradient Descent

من كشكولك

Gradient Descent هو Optimization Algorithm. فكرته: يعدّل الـ Parameters تدريجيًا لحد ما يقلل الـ Cost function.

تخيّل إنك فوق جبل وفي ضباب ومش شايف الوادي، لكن قادر تحس بميل الأرض، فهتنزل في اتجاه الانحدار ← ده بالظبط تشبيه الكتاب.

في ML: Cost function ← Gradient ← أعرف اتجاه زيادة الخطأ ← أتحرك في الاتجاه **العكسي** (فيقل الخطأ).

من كشكولك

الـ Gradient إيه؟

هو مجموعة الـ partial derivatives (المشتقات الجزئية) بالنسبة لكل Parameter. يعني بدل ما نسأل: لو غيّرت `θ1` شوية، الـ cost هتتغير قد إيه؟ نسأل السؤال ده لكل Parameter: `θ0 -> derivative` , `θ1 -> derivative` , ... `θn -> derivative`. ونجمعهم في Gradient vector.

$$\nabla_\theta \text{MSE}(\theta) = \left(\frac{\partial \text{MSE}}{\partial \theta_0}, \frac{\partial \text{MSE}}{\partial \theta_1}, \ldots, \frac{\partial \text{MSE}}{\partial \theta_n} \right)$$

من كشكولك

المهم: معادلة Gradient Descent:

$$\theta^{(\text{next step})} = \theta - \eta \nabla_\theta \text{MSE}(\theta)$$

وده قلب الـ GD. اقرأها كده: **القيمة الجديدة = القيمة الحالية − خطوة النزول**. الخطوة بتعتمد على: η (learning rate) وGradient (= اتجاه ومقدار تغيّر الـ cost).

إضافة

للـ MSE في الـ Linear Regression الـ Gradient vector له صيغة جاهزة (هنستخدمها في الكود جاي):

$$\nabla_\theta \text{MSE}(\theta) = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$$

لاحظ إن المعادلة بتستخدم **الداتا كلها** (`X` كاملة) ← ده بالظبط اسم Batch Gradient Descent اللي جاي بعد شوية.

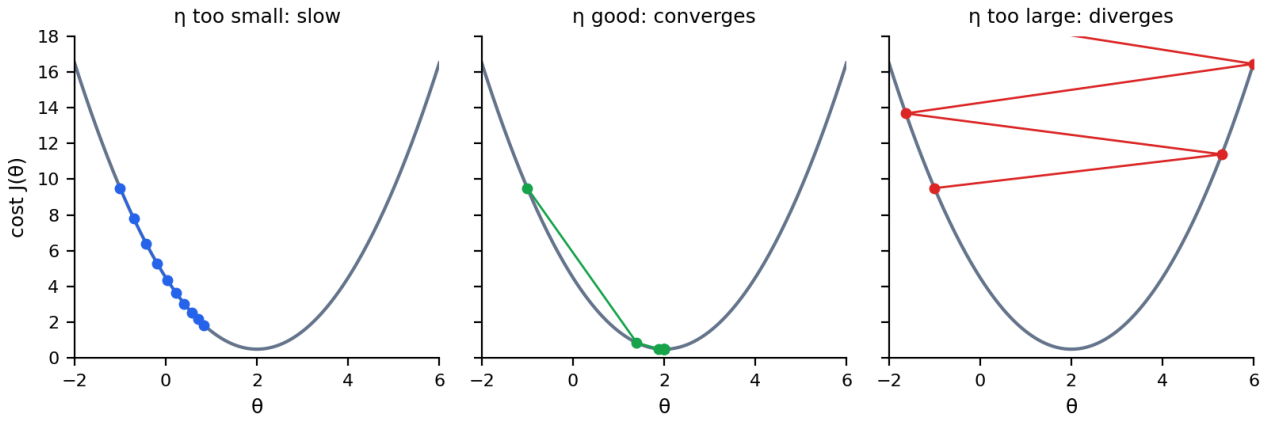
من كشكولك

الـ Learning Rate يعني إيه؟

هو حجم الخطوة اللي بتتحرك بيها في كل Iteration:

- لو **صغيرة جدًا** ← يوصل للـ minimum لكن ببطء شديد (تقارب بطيء).

• لو الخطوة **كبيرة** ← ممكن يعدّي الـ minimum وقد لا يستقر، ويمكن يبعد أكتر.



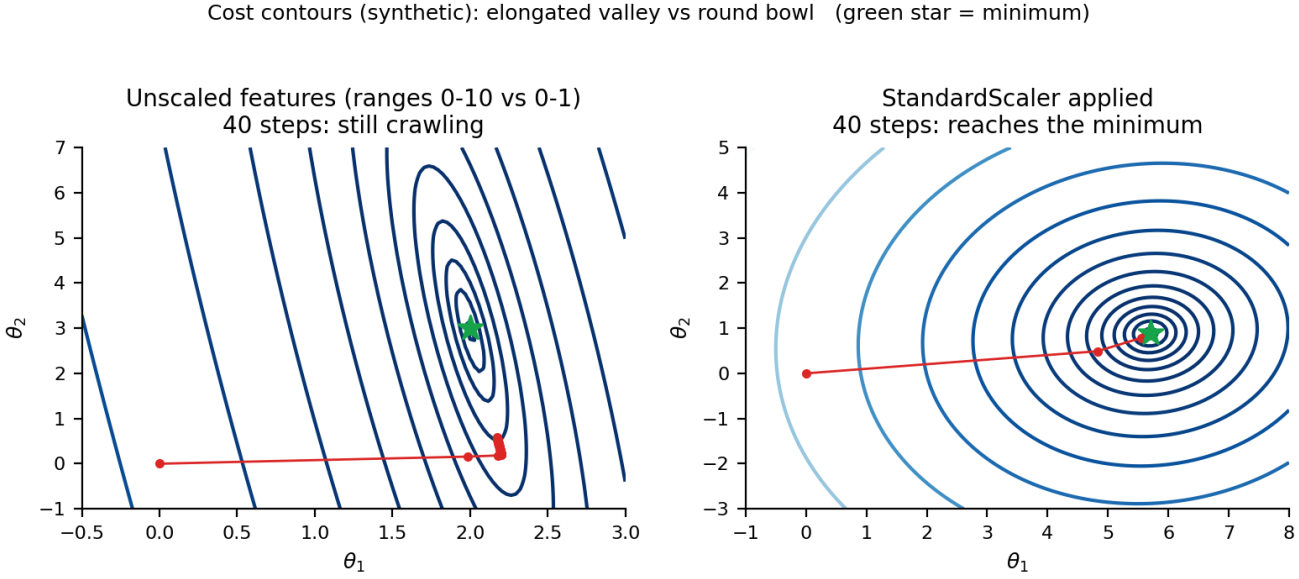
نفس الدالة ونفس نقطة البداية بـ 3 قيم لـ η: الصغيرة بتزحف، المناسبة بتوصل في خطوات قليلة، والكبيرة بتقفز لبعيد وتبعد عن الحل.

من كشكولك

ليه Scaling مهم جدًا مع Gradient Descent؟

لو عندي features بأحجام مختلفة جدًا، شكل الـ Cost surface يبقى زي **وادي طويل ورفيع** بدل ما يكون أقرب لوعاء متناسق، فالـ GD يتحرك بشكل غير مباشر ويحتاج وقت أطول للوصول للـ minimum.

لذلك مع GD: من المهم إن الـ features تكون على scales متقاربة، مثلًا باستخدام `StandardScaler` .



نفس عدد الخطوات (40) ونفس نسبة الـ learning rate: بدون scaling الخط الأحمر لسه بيزحف في الوادي، ومع StandardScaler وصل للنجمة الخضرا. (داتا synthetic)

إضافة

الـ Normal Equation **مش محتاجة** scaling، لكن أي حاجة بتستخدم Gradient Descent (بأنواعه + `SGDRegressor` + الـ Regularized models) **محتاجة**.

إمتى أوقف؟ الكتاب بيقول: خلّي عدد الـ epochs كبير، وأوقف لما الـ Gradient vector يبقى صغير جدًا (أصغر من قيمة صغيرة `ε` اسمها tolerance)، لأن ساعتها وصلت تقريبًا للـ minimum.

من كشكولك

الـ Convex Function

الكتاب بيتكلم عن Convex function: الـ `MSE` Cost function في الـ linear regression هي convex، وده مهم جدًا لأنه يعني إن عندنا Global Minimum واحد ولا يوجد local minima.

فلو استخدمنا Gradient Descent بشكل مناسب فهو **مضمون** إنه يقترب من الحل العالمي كلما استمر التدريب بما يكفي والـ learning rate مناسب. ← علشان كده GD مناسب للـ linear regression.

إضافة

تعريف convex ببساطة: لو رسمت خط مستقيم بين أي نقطتين على المنحنى، الخط ده عمره ما هينزل **تحت** المنحنى. شكلها وعاء (bowl) مفيش فيه حفر جانبية تعلّق فيها.

(الـ Cost functions بتاعة الـ Neural Networks مش convex، وهناك الـ local minima والـ plateaus مشكلة حقيقية. بس الـ Linear و Logistic Regression محظوظين.)

5 الـ Batch Gradient Descent

من كشكولك

في كل step: يستخدم الـ Training set **كله** لحساب الـ Gradient. يعني: كل الـ Training set ← احسب Gradient ← حدّث θ. وده بيخلّي كل step غالي لو الداتا ضخمة.

الـ epoch يعني إيه؟ مرور واحد كامل على الـ Training set. في Batch GD كل epoch معناها إنك تحسب الـ Gradient باستخدام كل البيانات مرة.

```
eta = 0.1          # learning rate
n_epochs = 1000
theta = np.random.randn(2, 1)              # random initialization

for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
print(theta)
```

Output (real run)

```
[[4.2151]
 [2.7701]]
```

من كشكولك

الفكرة: θ عشوائي في البداية، احسب Gradient، حدّث θ، كرّر، تقرب من أفضل θ. وفي النهاية وصل لنفس النتيجة اللي جابها الـ Normal Equation تقريبًا. وده شيء مهم: **طريقتين مختلفتين للتدريب ممكن تصلوا لنفس الحل**.

إضافة

قارن الـ Output ده بـ `theta_best` في القسم 3: نفس الأرقام تقريبًا (4.215 و 2.770). وتجربة على نفس الكود (نتايج حقيقية بعد 1000 epoch): `eta=0.02` وصل لـ `4.2144, 2.7707` (قريب جدًا بس أبطأ شوية)، لكن `eta=0.5` **فرّ** (أرقام بحجم 10^27)، و `eta=1.1` طلع `nan`. ده الـ divergence اللي في الرسمة اللي فاتت.

6 الـ Stochastic Gradient Descent (SGD)

من كشكولك

هنا الكتاب يحل مشكلة Batch GD: **بطيء** لأنه يستخدم كل البيانات في كل step.

في Stochastic Gradient Descent: كل step يختار instance **واحدة عشوائيًا** ويحسب Gradient بناءً عليها فقط.

- **الميزة:** كل step أسرع بكتير، ويمكن يتعامل مع Datasets ضخمة جدًا.
- **العيب:** المسار يصبح Noisy / Stochastic: الـ Cost قد تزيد وتنخفض بدل ما تنخفض بسلاسة.

من كشكولك

Learning Schedule في SGD

هنا SGD: الـ Randomness يعني إنه يفضل يتحرك حوالين الـ minimum بدل ما يستقر عليه. **الحل:** نقلّل الـ learning rate تدريجيًا.

في البداية learning rate كبير ← تقدّم بسرعة، ثم learning rate أصغر ← نقترب من الـ minimum. والدالة اللي بتحدد قيمة الـ learning rate في كل خطوة (step) تسمى Learning Schedule.

$$\eta(t) = \frac{t_0}{t + t_1}$$

```
n_epochs = 50
t0, t1 = 5, 50      # learning schedule hyperparameters (control the shape of eta)

def learning_schedule(t):
    return t0 / (t + t1)   # eta depends on the current training step number t
```

من كشكولك

دول قيمتين بيتحكموا في شكل الـ learning rate. والدالة بتحسب الـ learning rate بناءً على عدد خطوات التدريب الحالية (t).

من كشكولك

خلينا نفهم إحنا بنحل مشكلة إيه أصلًا في الأول: إنت عارف إن الـ Gradient descent بيحدّث الـ Parameters بـ `θ_new = θ_old − η ∇θ MSE`. هنا `η` هي learning rate يعني حجم الخطوة اللي بتتحرك بيها الـ Parameters.

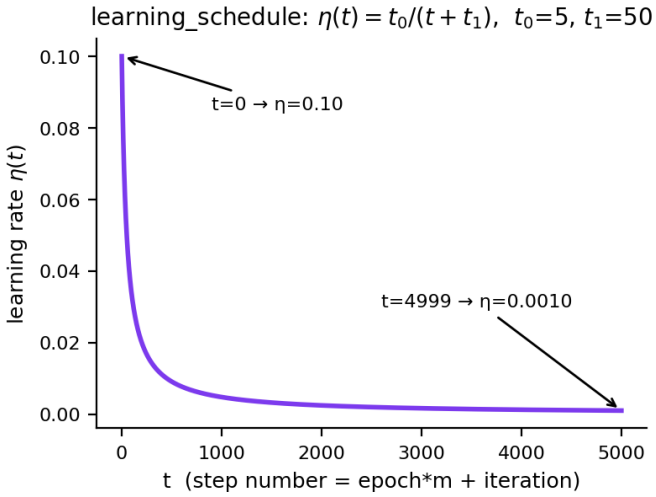
المشكلة: إنت مش عايز حجم الخطوة يفضل ثابت طول التدريب بالضرورة. في البداية نحتاج خطوات كبيرة نسبيًا عشان نقترب من الحل بسرعة، وبعد كده نحتاج خطوات أصغر عشان ما نفضلش نقفز حوالين الحل. وده دور الـ `learning_schedule()`. لاحظ كل ما `t` تزيد قيمة learning rate تقل.

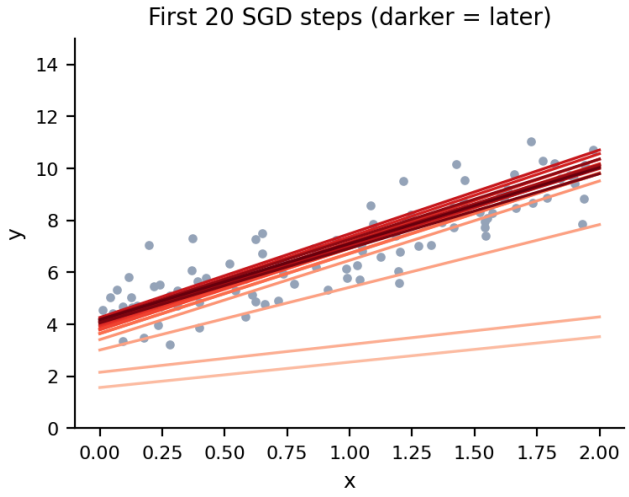
الخلاصة: الـ learning schedule هي دالة بتحدد إزاي الـ learning rate يتغيّر أثناء التدريب بدل ما تفضل ثابتة.

```
np.random.seed(42)
theta = np.random.randn(2, 1)      # initialization of the parameters
```

من كشكولك

في الشكل `(2, 1)` لأن المثال فيه feature واحدة (لكننا أضفنا عمود Bias باستخدام `X_b = add_dummy_feature(X)`). فبقى عندنا معاملين مطلوب تعلّمهم: `00` ← Bias / Intercept، و `01` ← وزن الـ feature (الوحيدة). إحنا بدأنا بقيم عشوائية، وبعد كده SGD هيحاول يحسّنها تدريجيًا.





شمال: الـ learning rate بيقل من 0.10 لحد 0.001 تقريبًا. يمين: أول 20 خطوة SGD (من تشغيل حقيقي): الخطوط الفاتحة بدأت بعيدة والغامقة قريبة من الحل.

من كشكولك

حلقة التدريب الأساسية (الكود)

في المثال السابق `m = 100` : فبنعمل `m` خطوات داخل كل epoch (هو عدد عينات التدريب). يعني: `50 epochs × 100 steps = 5000 updates`.

```
for epoch in range(n_epochs):
    for iteration in range(m):
        random_index = np.random.randint(m)        # pick a random sample index 0..m-1
        xi = X_b[random_index : random_index + 1]  # its features
        yi = y[random_index : random_index + 1]     # its real target
        gradients = 2 * xi.T @ (xi @ theta - yi)    # gradient from ONE sample only
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
print(theta)
```

Output (real run)

```
[[4.2108]
 [2.7486]]
```

من كشكولك

- `random_index` : يختار index عشوائيًا بين `0` و `m-1` . لو اختار العينة 24 إذن `xi` = Features الخاصة بالعينة 24، و `yi` = القيمة الحقيقية للعينة.
- `gradients` : حساب الـ Gradient لعينة واحدة بدل ما نحسب الـ Gradient باستخدام الـ dataset بالكامل (Batch). هنا بنحسب الـ prediction (`xi @ theta`) ونقارنه بـ `yi` .
- `theta = theta - eta * gradients` : تحديث نفس الـ Parameters. معناها: خد القيم الحالية للـ Parameters وحرّكها في **عكس** اتجاه الـ Gradient بمقدار تحدده learning rate.

من كشكولك

ليه لازم العينة تكون iid أو نعمل shuffle؟

IID اختصار (Independent and Identically Distributed) أي إن العينات **مستقلة** عن بعضها ومأخوذة من **نفس التوزيع** الاحتمالي.

تخيّل إن بياناتك مرتبة حسب الـ label: كل عينات الفئة A ثم B ثم C. لو دربت SGD بالترتيب ده، هتكون تحديثاتك في البداية متأثرة بعينات A وبعدها هتنتقل لـ B. ده ممكن يجعل مسار التدريب **غير متوازن** ويؤثر على التقارب.

لذلك ممكن نعمل Shuffle للبيانات مع تحريك الـ Features والـ Targets معًا للحفاظ على تطابق كل عينة مع إجابتها.

من كشكولك

استخدام SGDRegressor بدل كتابة الكود يدويًا

```
from sklearn.linear_model import SGDRegressor
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None,
                       eta0=0.01, n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel())
print(sgd_reg.intercept_, sgd_reg.coef_)
```

```
Output (real run)
[4.2128] [2.7727]
```

من كشكولك

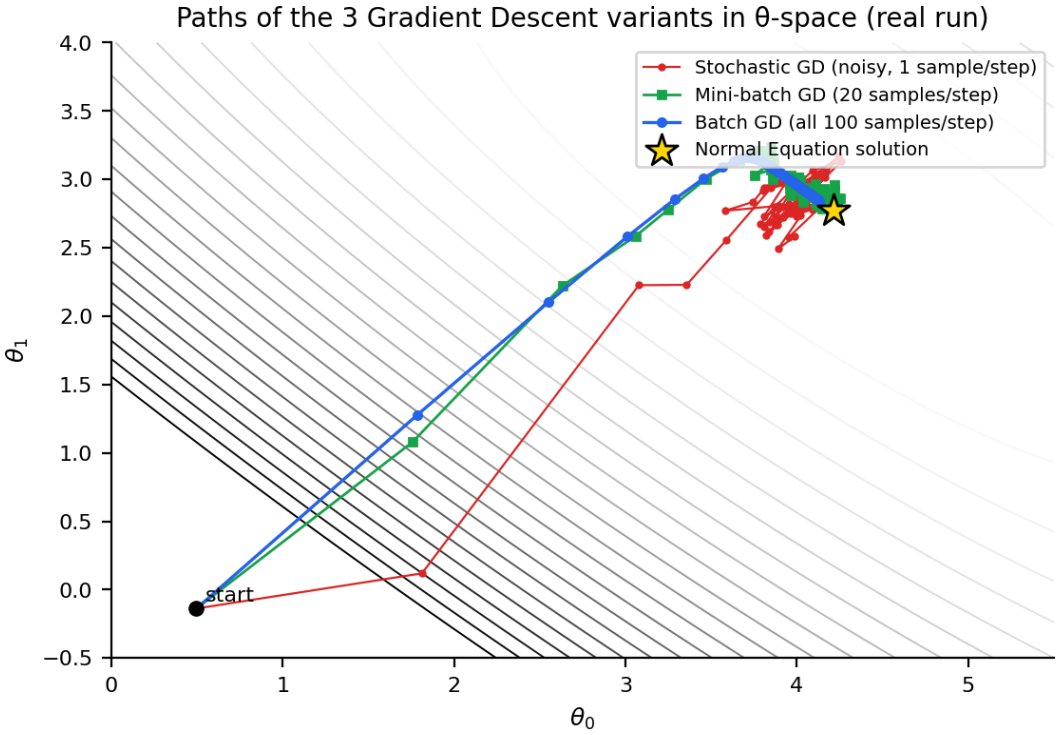
- `max_iter` : أقصى عدد من epochs (المقدار الأقصى للاستمرار في التدريب).
- `penalty=None` : عدم استخدام Regularization.
- `eta0` : قيمة الـ learning rate الابتدائية.
- `n_iter_no_change` : عدد الـ epochs المتتالية التي يراقب خلالها عدم تحقق تحسّن كافي قبل إيقاف التدريب.

إضافة

`tol=1e-5` معناها: لو الـ loss ما نزلتش بأكتر من `0.00001` على مدى `n_iter_no_change=100` epoch متتالية، يوقف التدريب. وده Early stopping مدمج (هتكلم عنه في قسم 12).

ملحوظة: SGD بيحتاج feature scaling زي أي GD، وفي بيانات حقيقية لازم تحط `StandardScaler` قبله في pipeline.

ميزة تانية للعشوائية: لو الـ cost مش convex (زي الـ Neural Networks) الاهتزاز بتاع SGD ممكن يساعده يطلع من local minima، بس بعد كده محتاج learning schedule عشان يستقر. الكتاب بيشبّهها بـ simulated annealing.



مسارات الأنواع التلاتة في فضاء (θ0, θ1) من تشغيل حقيقي: Batch (أزرق) ناعم، Mini-batch (أخضر) فيه شوية اهتزاز، SGD (أحمر) مشوّش جدًا حوالين الحل. النجمة = حل الـ Normal Equation.

7 الـ Mini-Batch Gradient Descent

من كشكولك

حل وسط بين Batch و SGD: نستخدم **مجموعات صغيرة** من الـ instances (تسمى Mini-batches) في كل step.

ميزته: يحقق توازن بين استقرار Batch وسرعة SGD؛ ومن مميزاته إنه يستفيد من تسريع الـ hardware للعمليات الـ matrix خصوصًا مع الـ GPUs.

إضافة

جدول المقارنة الكامل بين طرق التدريب (من الكتاب بتصرف):

الخوارزمية	كبر m (عدد العينات)	كبر n (عدد الـ features)	Hyperparameters	محتاج Scaling؟
Normal Equation	سريعة (خطية في m)	**بطيئة** (حوالي O(n^2.4..3))	0	لأ
SVD (LinearRegression)	سريعة	بطيئة (حوالي O(n^2))	0	لأ
Batch GD	**بطيئة**	سريعة	2 (η و epochs)	أيوه
Stochastic GD	سريعة (تقدر تشتغل out-of-core)	سريعة	≤ 2 (+ schedule)	أيوه

الخوارزمية	كبر m (عدد العينات)	كبر n (عدد الـ features)	Hyperparameters	محتاج Scaling؟
Mini-batch GD	سريعة	سريعة	حجم الـ batch (+ 2 ≤)	أيوه

وكلهم بعد التدريب نفس النتيجة تقريبًا في الـ Linear Regression. الفرق في السرعة والذاكرة وإمكانية التعامل مع داتا ضخمة.

8 الـ Polynomial Regression

من كشكولك

المشكلة: الـ Linear regression بيفترض إن العلاقة بين الـ features والـ Target **خطية**، يعني تقريبًا `y = ax + b`. لكن ماذا لو البيانات شكلها منحنى؟ مثل العلاقة الحقيقية `y = ax² + bx + c`: هنا الخط المستقيم مش هيقدر يمثّل البيانات كويس.

الكتاب بيحل المشكلة بإننا نحوّل الـ features قبل ما نديها للـ linear Regression.

الفكرة: نضيف Features جديدة عبارة عن Powers للـ Features الأصلية. مثال: عندنا `x` نحوّلها لـ `x` و `x²`، وبعدين نتدرّب linear regression:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$$

هنا الموديل لسه linear في الـ Parameters، رغم إن العلاقة مع `x` نفسها أصبحت **غير خطية**. ← Polynomial Regression = تضيف polynomial features ثم نستخدم linear regression.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

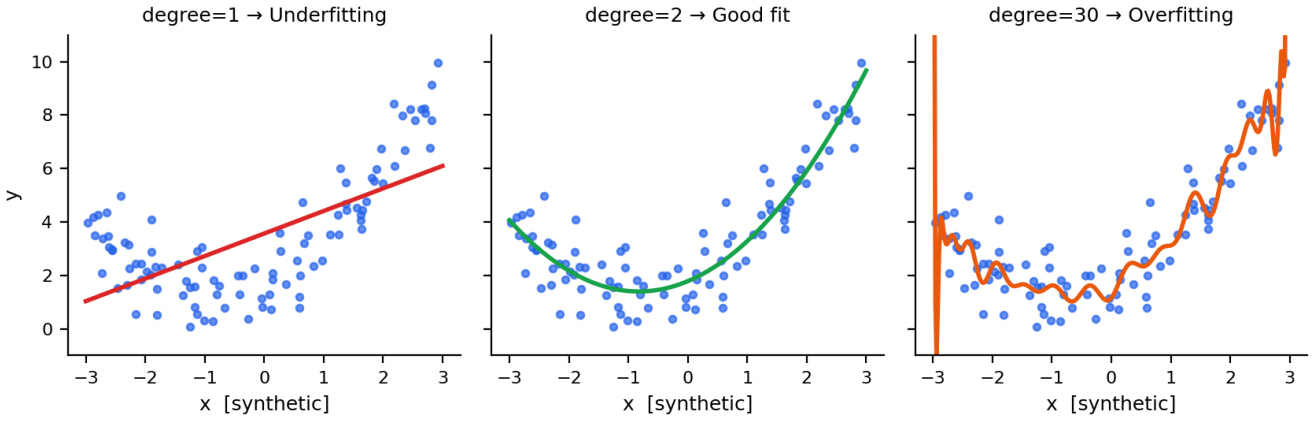
poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)

lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
print(X[0], X_poly[0])
print(lin_reg.intercept_, lin_reg.coef_)
```

```
Output (real run)
[-0.7528] [-0.7528  0.5666]
[1.7813] [[0.9337 0.5646]]
```

من كشكولك

- `include_bias=False` : يعني لا تضيف عمود الـ `1` الخاص بـ bias، لأن linear Regression هيتعامل مع intercept بنفسه.
- `degree=2` : معناها السؤال الأقصى للموديل: هيحسب لك، لو عندك feature واحدة `(x)` : الـ `degree=2` يخلق لنا الأهمية دي: `x^1` (الـ Feature الأصلية)، `x^2` (مربع الـ feature). باختصار الـ degree يحدد إحنا عايزين نطلع لأي درجة من الأُسس (تاني، تالت...).
- ثم بتقول `lin_reg.fit(X_poly, y)` : الموديل يتعلّم $\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$.



نفس الداتا (synthetic، علاقتها الحقيقية تربيعية) بـ 3 درجات: درجة 1 مش قادرة تمسك المنحنى، درجة 2 مظبوطة، درجة 30 بتلف حوالين كل نقطة.

من كشكولك

Polynomial Features مع أكتر من feature

لو عندي features: `a, b` واستخدمت `degree=3` مش هيطلع فقط a, b, a^2, b^2, a^3, b^3 لكن يضيف أيضًا Combinations مثل: ab, a^2b, ab^2 . وبالتالي يقدر يكتشف **علاقات بين الـ features نفسها** (وهو شيء الـ linear regression العادي مش بيقدر يعمله).

Combinatorial Explosion

كل ما تزوّد الـ degree أو عدد الـ features، عدد الـ features الجديدة ممكن يزيد **بشكل ضخم**. Polynomial Regression ممكن تكون قوية، لكن الـ degree العالية جدًا ممكن تنتج عددًا هائلًا من الـ Features.

إضافة

الصيغة (عدد الـ features بعد التحويل، بتشمل الـ bias term):

$$\text{number of features} = \frac{(n+d)!}{d!\ n!}$$

على سبيل المثال: `n=10` و `d=3` يطلع **286** (منهم الـ bias يعني 285 feature فعلية). لكن `n=100` و `d=3` يطلع حوالي **176 ألف**. شوف كمان أسماء الـ features اللي `PolynomialFeatures(3)` بتطلعها لـ `a` و `b` :

```
PolynomialFeatures(3, include_bias=False).fit(np.zeros((1, 2))).get_feature_names_out(["a", "b"])
```

Output (real run)

```
['a' 'b' 'a^2' 'a b' 'b^2' 'a^3' 'a^2 b' 'a b^2' 'b^3']
```

نصيحة عملية: استخدم دايمًا `StandardScaler` بعد `PolynomialFeatures` (لأن `x^30` أرقامها ضخمة جدًا وعمل scale مهم للتدريب).

9 الـ Overfitting والـ Underfitting والـ Learning Curves

من كشكولك

إيه اللي هيحصل لو الـ degree عالي جدًا؟ هنا ندخل على Underfitting و Overfitting: الكتاب جرّب Polynomial degree كبير جدًا، فالموديل حاول يتبع بيانات التدريب **نقطة بنقطة** وأصبح المنحنى متعرّج جدًا. النتيجة:

- linear model ← Underfitting.
- High-degree model ← Overfitting.

إزاي أعرف إن الموديل Overfit أو Underfit؟ الكتاب يرجّع لفكرة Cross-Validation:

الحالة	Training performance	Validation performance
Overfitting	ممتاز	سيء
Underfitting	سيء	سيء

إضافة

وده بالظبط اللي شفناه في شابتر 2: الـ Decision Tree طلع `RMSE = 0` على الـ training (ممتاز) لكن حوالي `66,868` في الـ cross-validation (سيء) ← Overfitting. والـ Linear Regression كان سيء في الاتنين (`68,688` على الـ training) ← Underfitting.

العلاج بسرعة: Underfitting ← موديل أقوى / features أحسن / قلّل القيود (الـ regularization). Overfitting ← بسّط الموديل / قيّده (Regularization) / هات داتا أكتر.

الـ Learning Curves

من كشكولك

دي أداة إضافية لمعرفة هل الموديل underfit ولا overfit.

الفكرة: نتابع Training error و Validation error مع **زيادة حجم الـ Training set**. الكتاب بيوفّر `learning_curve()` وهي بترجع: `train_sizes, train_scores, valid_scores`. ونقدر نرسم الـ errors مقابل حجم الـ training data.

```python
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40), cv=5,
    scoring="neg_root_mean_squared_error")

train_errors = -train_scores.mean(axis=1)    # negate: scores are NEGATIVE rmse
valid_errors = -valid_scores.mean(axis=1)
```

إضافة

انتبه للسالب: زي ما شرحنا في شابتر 2 (`cross_val_score`، `scoring="neg_root_mean_squared_error"`) بترجع قيم **سالبة**، فلازم نحط علامة `-` قدامها عشان نرجّع الـ RMSE الحقيقي.

من كشكولك

learning curve للـ linear model: لو لقيت:

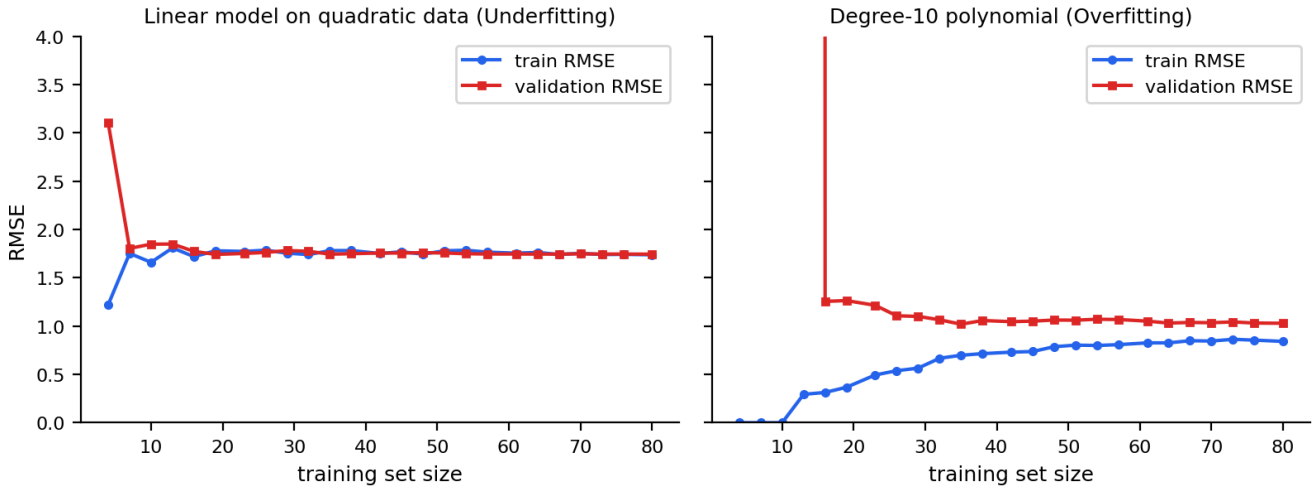
1. Training error يبدأ منخفض ثم يزيد ويستقر.
2. Validation error يبدأ عاليًا ثم ينخفض.
3. في النهاية المنحنيان قريبان من بعض ومرتفعان نسبيًا.

← ده شكل يوجّه إلى Underfitting.

ولو لقيت:

1. Training error منخفض جدًا.
2. Validation error أعلى.
3. وفيه Gap واضح بينهما.

← ده من علامات Overfitting. والكتاب بيذكر إن زيادة الـ Training data قد تساعد في تقليل الـ Gap.



من تشغيل حقيقي على داتا تربيعية (synthetic): شمال: الموديل الخطي، المنحنيين اتلزقوا عند RMSE حوالي 1.74 ومفيش فايدة من داتا أكتر (Underfitting). يمين: درجة 10، الـ train حوالي 0.84 والـ validation حوالي 1.03 والفجوة بتضيق مع زيادة الداتا (Overfitting).

إضافة

القاعدة اللي تنفعك في المراجعة: لو المنحنيين لزقوا في بعض عند خطأ عالي، يبقى **داتا أكتر مش هتفيد** (الموديل نفسه ضعيف). لو فيه فجوة، يبقى **داتا أكتر** غالبًا هتفيد.

10 الـ Generalization Error والـ Bias/Variance Trade-off

من كشكولك

Generalization Error (خطأ التعميم): هو باختصار معدل خطأ الموديل لما يشتغل على بيانات جديدة تمامًا لسه ما شافهاش قبل (Unseen Data). كل ما الخطأ ده كان قليل كل ما الموديل كان "أذكى" وقدرته على التعميم أفضل.

الخطأ الكلي في أي موديل بيتكوّن من 3 حاجات رئيسية:

من كشكولك

① الـ Bias (الانحياز أو "العناد")

- **معناه:** الموديل بيبقى بسيط زيادة عن اللزوم وعنده قناعات ثابتة مش بيغيّرها مهما حصل.
- **مثال:** زي ما تحاول ترسم خط مستقيم عشان توصل بيه نقاط ماشية في الأساس على شكل قوس (Quadratic). الموديل رافض يفهم التقوّس وعنده "انحياز" للخط المستقيم.
- **النتيجة:** Underfitting (قصور في التعلّم): الموديل غبي ومش فاهم حاجة لا في التدريب ولا في الاختبار (الجديد).

② الـ Variance (التباين أو "التأثّر السريع")

- **معناه:** الموديل بيبقى معقّد زيادة عن اللزوم وحسّاس جدًا لأي تغيير طفيف في بيانات التدريب.
- **مثال:** لو جبت polynomial بدرجة عالية جدًا (High-degree Polynomial) وخلّيته يعدّي بكل نقطة بدقة مرعبة، لو غيّرت نقطة واحدة بس في بيانات التدريب، شكل المنحنى ككل هيتغيّر بطريقة غريبة.
- **النتيجة:** Overfitting: الموديل حفظ بيانات التدريب بالعرف بكل الـ noise اللي فيها، لكنه لما يشوف بيانات جديدة بيفشل تمامًا.

③ الـ Irreducible Error (الخطأ اللي لا يمكن إزالته)

- **معناه:** ده الخطأ المتبقي بسبب "الضوضاء" (Noise) الأصلي الموجود في الحياة نفسها وفي طريقة تجميع البيانات (زي خطأ في قراءة جهاز قياس، أو بيانات مدخلة غلط من البداية).
- **الحل:** مهما كان الموديل عبقري مش هيقدر يحل ده؛ الحل الوحيد إنك تنضّف الداتا بتاعتك (Data cleaning) وتظبطها.

إضافة

$$\text{Generalization Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

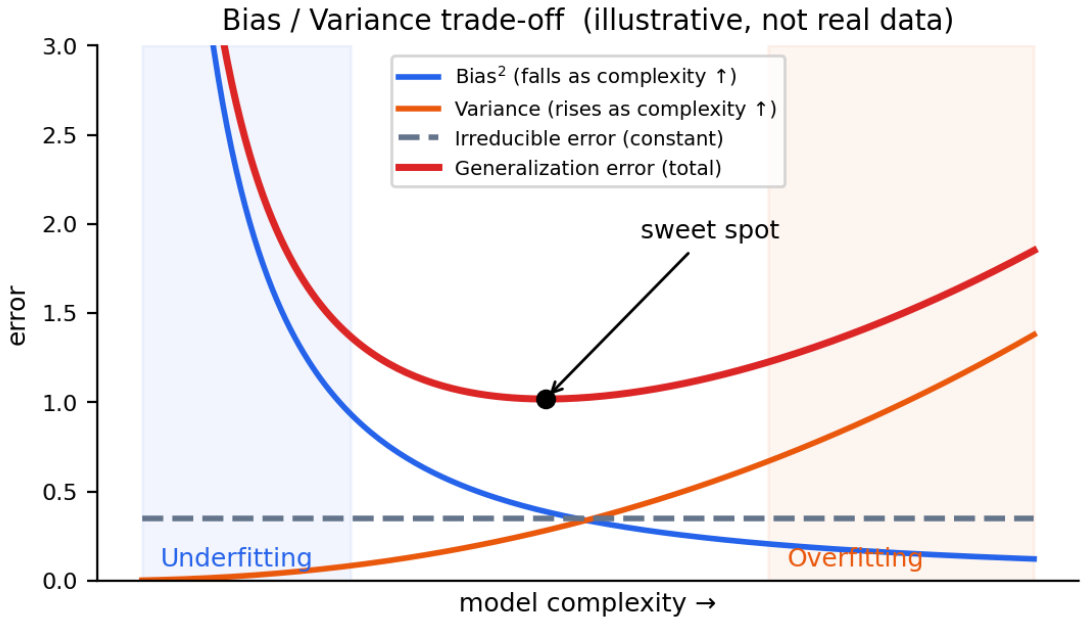
وده التعبير الرياضي اللي في الكتاب: الأجزاء التلاتة بتتجمّع. اتنين منهم (Bias و Variance) إحنا نقدر نأثر عليهم باختيار الموديل، والتالت ثابت.

من كشكولك

إيه هو الـ Bias/Variance Trade-off (معضلة الشدّ والجذب)؟

العلاقة بين الـ Bias والـ Variance زي "الأرجوحة" (See-saw): مينفعش الاتنين يبقوا قليلين في نفس اللحظة لوحدهم، لازم توازن بينهم:

- لما **تعلّي** تعقيد الموديل (Model Complexity ↑): الـ Bias ↓ (الموديل يطابق بناء وتقرّب من تفاصيل الداتا) والـ Variance ↑ (الموديل يبقى حسّاس ويتغير بسهولة).
- لما **تقلّل** تعقيد الموديل (Model Complexity ↓): الـ Variance ↓ (الموديل يبقى مستقر ومش بيتأثر بأي هوا) والـ Bias ↑ (الموديل يبقى سطحي جدًا وعاجز عن فهم الأشياء).
- **الـ Trade-off:** شغلتك كمهندس إنك تقف في "النقطة" المناسبة، عشان توصل لأقل مجموع ممكن (Bias + Variance). ساعتها يتركّب أفضل موديل ممكن يحقق أقل Generalization error.



الرسمة توضيحية (illustrative): ناحية الشمال Bias عالي (Underfitting)، ناحية اليمين Variance عالي (Overfitting)، والخط الأحمر (الخطأ الكلي) أقل ما يمكن في النص.

إضافة

	Bias عالي	Variance عالي
الموديل	بسيط زيادة	معقّد زيادة
Training error	عالي	منخفض جدًا
Validation error	عالي (قريب من الـ training)	عالي (بعيد عن الـ training = gap)
الحالة	Underfitting	Overfitting
العلاج	موديل أقوى أو features أحسن	داتا أكتر أو Regularization أو موديل أبسط

11 الـ Regularization والـ Ridge Regression

من كشكولك

Regularization: هو إنك لو عندك overfitting ممكن تقيّد الموديل. **الفكرة:** بدل ما نسمح للـ model weights تكون بأي قيم، نفرض عليها قيودًا عشان الموديل ما يبقاش معقّد زيادة عن اللزوم.

وفي الـ linear model الكتاب يدخّل 3 أنواع: ① Ridge ② Lasso ③ Elastic Net

من كشكولك

Ridge Regression

`Ridge = linear Regression + L2 Regularization`. نضيف Penalty للـ `MSE` تعتمد على **مربّعات** الـ weights:

$$J(\theta) = \text{MSE}(\theta) + \frac{\alpha}{m}\sum_{i=1}^{n}\theta_i^2$$

الهدف: الموديل مش بس يقلّل خطأ `MSE`، لكن كمان يحاول يخلّي الـ weights صغيرة.

من كشكولك

- `alpha` ده Hyperparameter بيتحكم في قوة الـ Regularization.
- `alpha = 0` ← linear Regression عادي.
- `alpha` أعلى ← Regularization أقوى ← weights أصغر ← موديل أبسط.
- لو `alpha` كبيرة جدًا الـ weights تقترب من الصفر، وبالتالي التنبؤ يبقى أقرب لخط **مسطّح** حوالين متوسط البيانات.

مهم: الـ Bias / Intercept (`θ0`) لا يتم الـ Regularize له في Ridge حسب صيغة الكتاب (الجمع بيبدأ من `i=1`).

من كشكولك

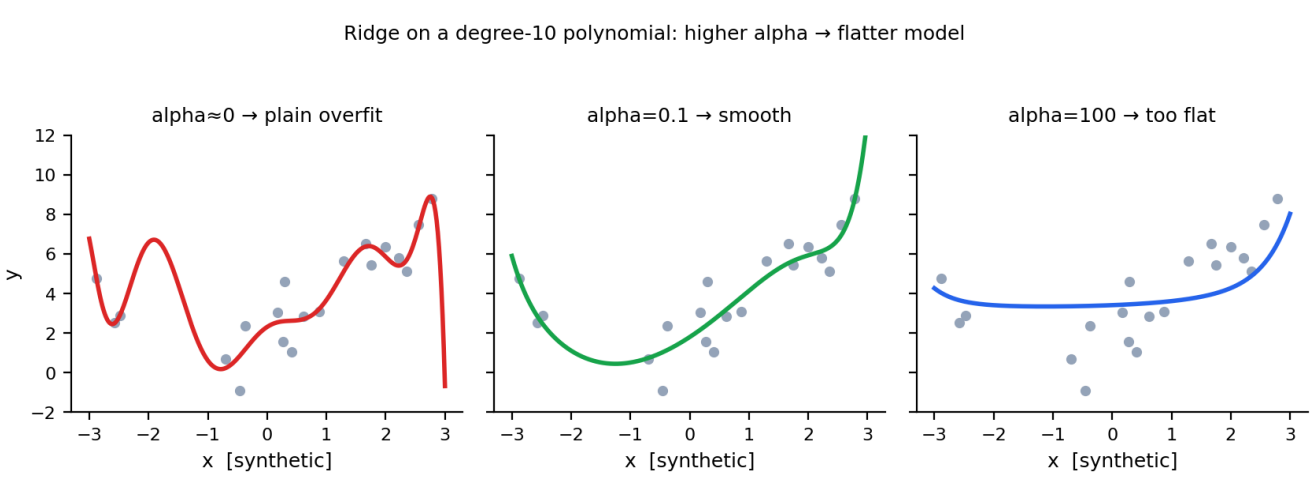
ليه Scaling مهم مع Ridge؟

الـ Regularization تعتمد على **حجم** الـ weights، لذلك اختلاف الـ Feature scales يؤثر على العقوبة. لذلك الكتاب يقول: لازم Scale the data before Ridge regression (وهو أمر صحيح لمعظم الـ Regularized models).

من كشكولك

Polynomial Regression مع Ridge

ممكن نعمل: `Polynomial Features` ← `Standard Scaler` ← `Ridge`. وكل ما `alpha` تزيد: الـ weights ↓، الـ Model يصير flatter، الـ Bias ↑ والـ Variance ↓.



نفس الـ pipeline (degree=10 + StandardScaler + Ridge) بـ 3 قيم alpha على 20 نقطة (synthetic): alpha≈0 اتعلّم التعرّجات، alpha=0.1 معقول، alpha=100 مسطّح زيادة (Underfitting).

Ridge في Scikit-learn

```
from sklearn.linear_model import Ridge
ridge_reg = Ridge(alpha=0.1, solver="cholesky")
ridge_reg.fit(X, y)
print(ridge_reg.predict([[1.5]]))
```

Output (real run)

```
[8.3659]
```

من كشكولك

- هنا: `Ridge` + `alpha` = Regularized linear regression؛ والـ `solver=` مجرد طريقة لحل المسألة رياضيًا في هذا المثال.
- الكتاب بيقول إنك تقدر تستخدم أيضًا `SGDRegressor(penalty="l2")` للحصول على نوع L2 Regularization نفسه بأسلوب SGD.

من كشكولك

RidgeCV

بدل ما تختار `alpha` يدويًا: Scikit-learn عندها `RidgeCV` : تعمل Ridge Regression وتختار الـ Hyperparameter باستخدام Cross-Validation. الكتاب بيقول إنها تقريبًا مثل `GridSearchCV` لكن مخصّصة ومحسّنة لـ Ridge.

```
from sklearn.linear_model import RidgeCV
ridge_cv = RidgeCV(alphas=[0.1, 1.0, 10.0])
ridge_cv.fit(X, y.ravel())
print(ridge_cv.alpha_)
```

Output (real run)

```
0.1
```

12 الـ Lasso والـ Elastic Net والـ Early Stopping

من كشكولك

Lasso Regression

Lasso هو Regularized linear regression أيضًا، لكن يستخدم L1 Regularization بدلًا من L2. يعني Penalty تعتمد على **القيم المطلقة** للـ weights (وليس مربّعاتها).

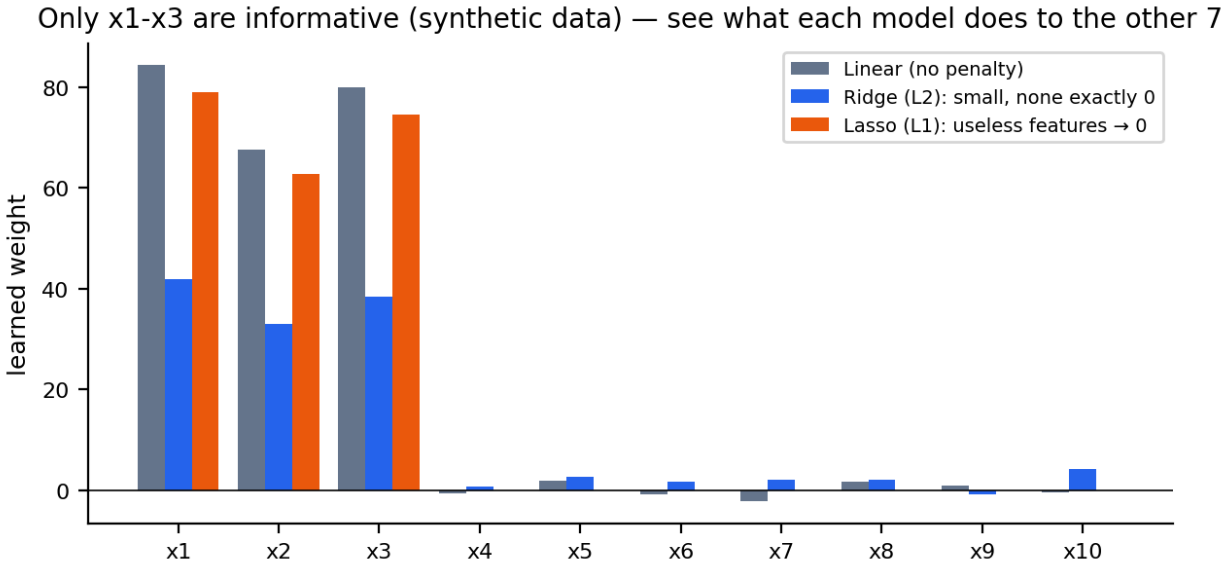
$$J(\theta) = \text{MSE}(\theta) + \alpha \sum_{i=1}^{n} |\theta_i|$$

أهم ميزة عملية لـ Lasso: يمكنه جعل بعض الـ weights تساوي **صفرًا** تمامًا، وبالتالي تقوم تلقائيًا بعملية Feature selection (يعني Features غير المهمة ممكن تختفي فعليًا من الموديل).

من كشكولك

وده فرق مهم جدًا: Ridge ← weights صغيرة. Lasso ← بعض الـ weights ممكن تصبح **صفر**.

```
from sklearn.linear_model import Lasso
lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
```



من تشغيل حقيقي (synthetic): 10 features منهم 3 بس مفيدين. Ridge صغّر كل الأوزان لكن ما صفّرش ولا واحد. Lasso صفّر 7 features من 10 (الـ 7 الغير مفيدين بالظبط) وسايب الـ 3 المفيدين.

انتبه / تصحيح

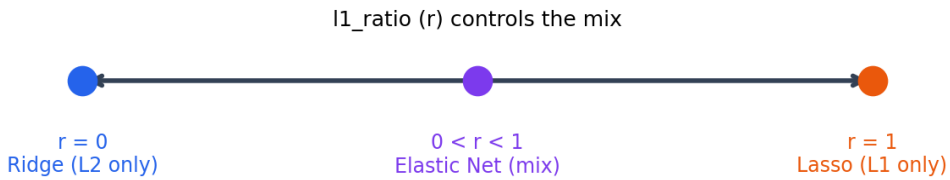
ثوابت الـ Penalty: شكل المعادلات (Ridge/Lasso/Elastic) اللي فوق هو الصيغة المفاهيمية. الثابت اللي بيضرب الـ alpha (زي `1/m` أو `2`) بيختلف من كتاب لتاني ومن مكتبة لمكتبة، فمتتوقعش إن نفس قيمة `alpha` تدي نفس النتيجة في أدوات مختلفة. الفكرة نفسها ما بتتغيرش.

من كشكولك

Elastic Net

هي **حل وسط** بين Ridge و Lasso وتستخدم مزيجًا من `L1 + L2` . وفيه Hyperparameter اسمه `r` (في sklearn اسمه `l1_ratio`) يتحكّم في نسبة المزج.

- `r = 0` ← Ridge.
- `r = 1` ← Lasso.
- وبالتالي `0 < r < 1` مزيج بينهم.



الـ Elastic Net بتتحكم فيها بـ r: من Ridge الصافي لـ Lasso الصافي.

من كشكولك

- Default ← Ridge جيد.
- لو تتوقع عددًا قليلًا فقط من features مفيدة ← Elastic Net / Lasso.
- يفضّل Elastic Net على Lasso غالبًا عندما تكون Features كثيرة جدًا أو بينها Correlation قوي (لأن Lasso قد يتصرف بشكل غير مستقر في بعض هذه الحالات).

إضافة

$$J(\theta) = \text{MSE}(\theta) + r\alpha \sum_{i=1}^{n} |\theta_i| + (1 - r)\alpha' \sum_{i=1}^{n} \theta_i^2$$

```
from sklearn.linear_model import ElasticNet
elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic_net.fit(X, y)
```

الموديل	النوع	بيعمل إيه؟	استخدمه لما...
Ridge	L2	يصغّر كل الأوزان	الحالة الافتراضية، كل الـ features مفيدة شوية
Lasso	L1	يصفّر أوزان بعض الـ features	عندك features كتير وقليل منها هو المفيد
Elastic Net	L1 + L2	مزيج	features كتير أو مرتبطة ببعض

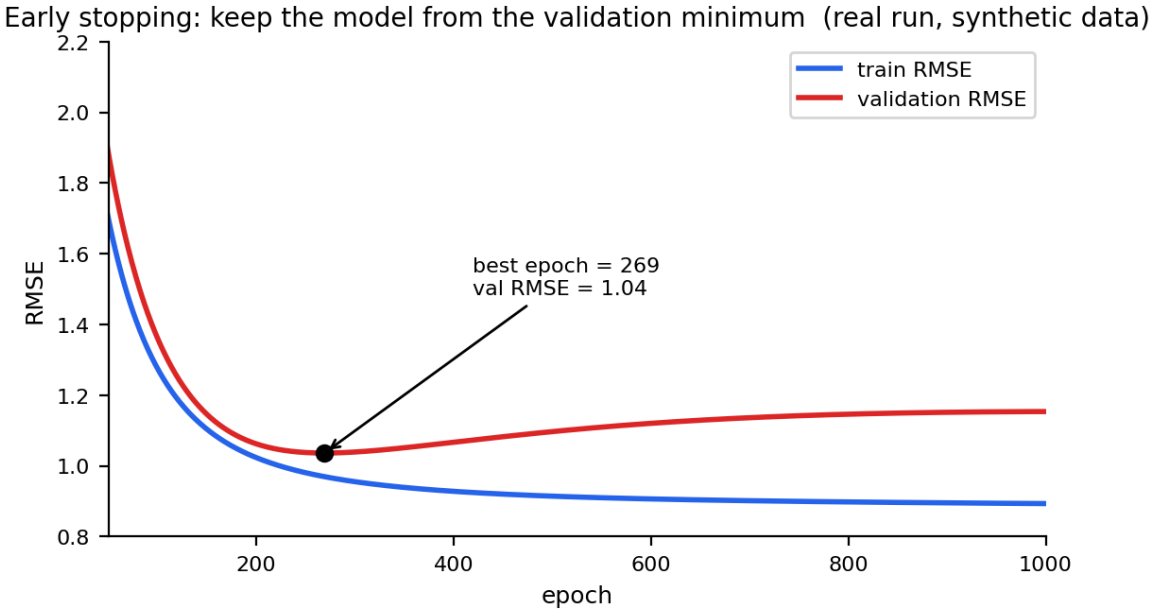
Early Stopping

من كشكولك

دي طريقة أخرى للـ Regularization خصوصًا مع Iterative Learning زي Gradient Descent.

الفكرة: وقف التدريب عندما يصل الـ Validation error لأقل نقطة، قبل ما يبدأ يزيد (overfitting).

يعني: ↑ Epochs ← ↓ Training error باستمرار تقريبًا، ↓ Validation error حتى minimum ثم يبدأ يزيد ← ↑ overfitting.



من تشغيل حقيقي (synthetic، degree=20): الـ train بيفضل ينزل، لكن الـ validation بيوصل لأقل نقطة عند epoch حوالي 269 وبعدها بيبدأ يطلع شوية (بداية الـ overfitting). الفرق صغير في الداتا دي، لكن الفكرة هي هي.

من كشكولك

Early stopping code

```
from copy import deepcopy
from sklearn.metrics import root_mean_squared_error

sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
best_valid_rmse = float("inf")
best_epoch = 0

for epoch in range(1000):
    sgd_reg.partial_fit(X_train_prep, y_train)      # one epoch, incrementally
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = root_mean_squared_error(y_valid, y_valid_predict)
    if val_error < best_valid_rmse:                 # better than any previous result
        best_valid_rmse = val_error
        best_epoch = epoch
        best_model = deepcopy(sgd_reg)              # keep a copy of the best state

print(best_epoch, best_valid_rmse)
```

Output (real run)

```
best epoch = 269, best validation RMSE = 1.0365
```

من كشكولك

- `partial_fit()` بدل `fit()` عشان يتعلّم incrementally خطوة بخطوة.
- بعد كل epoch: `y_valid_predict = sgd_reg.predict(X_valid_prep)` ثم نحسب `val_error`.
- ولو الـ Validation RMSE أقل من أي نتيجة سابقة: `best_valid_rmse = val_error` و `best_model = deepcopy(sgd_reg)`. يعني احتفظ بنسخة من أفضل حالة وصل لها الموديل حتى الآن.
- **نقطة مهمة:** في كود الكتاب ده لا يتم إيقاف التدريب فعليًا؛ هو فقط يحتفظ بأفضل موديل، وبعد انتهاء التدريب تستطيع الرجوع إليه.

من كشكولك

deepcopy vs clone

دي نقطة صغيرة لكن مهمة لو بتكتب كود:

- `deepcopy(sgd_reg)` ← ينسخ Hyperparameters + Learned Parameters.
- `sklearn.base.clone()` ← ينسخ **فقط** الـ Hyperparameters وليس الـ Learned parameters.

```
from sklearn.base import clone
print(hasattr(clone(sgd_reg), "coef_"))      # does the clone have learned weights?
print(hasattr(deepcopy(sgd_reg), "coef_"))   # does the deepcopy?
```

Output (real run)

```
False
True
```

إضافة

يعني لو عملت `clone` للموديل اللي اتدرّب وطلبت منه `predict` هيرفض (لأنه رجع "موديل جديد لسه ما اتدرّبش")، أما `deepcopy` فهو اللي بيحفظ حالة التدريب.

وفي بعض إصدارات الكود ممكن تلاقي `mean_squared_error` بدل `root_mean_squared_error` ؛ المقارنة `val_error < best` تظل صحيحة (لأن الجذر دالة تصاعدية) لكن الأرقام المطبوعة بتبقى MSE مش RMSE.

13 الـ Logistic Regression

من كشكولك

هنا الفصل ينتقل من Regression إلى Classification.

- يستخدم لتقدير **احتمال** انتماء الـ instance إلى الـ Positive class. مثلًا لو `Spam probability = 0.87`.
- لو الاحتمال أكبر من الـ Threshold (غالبًا 50% افتراضيًا) ← `1` ← Positive. غير كده `0` ← Negative.

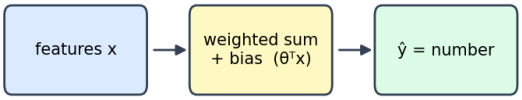
من كشكولك

الموديل أولًا يعمل: Weighted sum + Bias، يعني `t = θ^T x`:

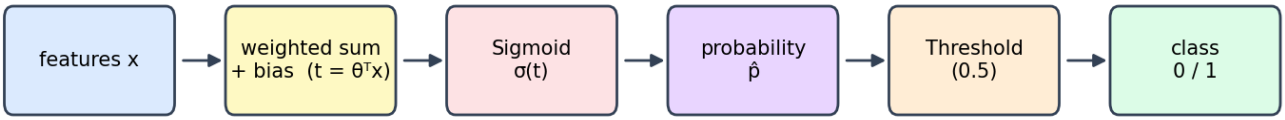
$$t = \theta^T \mathbf{x}$$

لكن لا يخرج `t` مباشرةً كما في linear regression؛ بدل ذلك يمرّرها إلى sigmoid / Logistic function:

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$



Linear Regression



Logistic Regression: نفس الـ weighted sum لكن بعده Sigmoid ثم Threshold

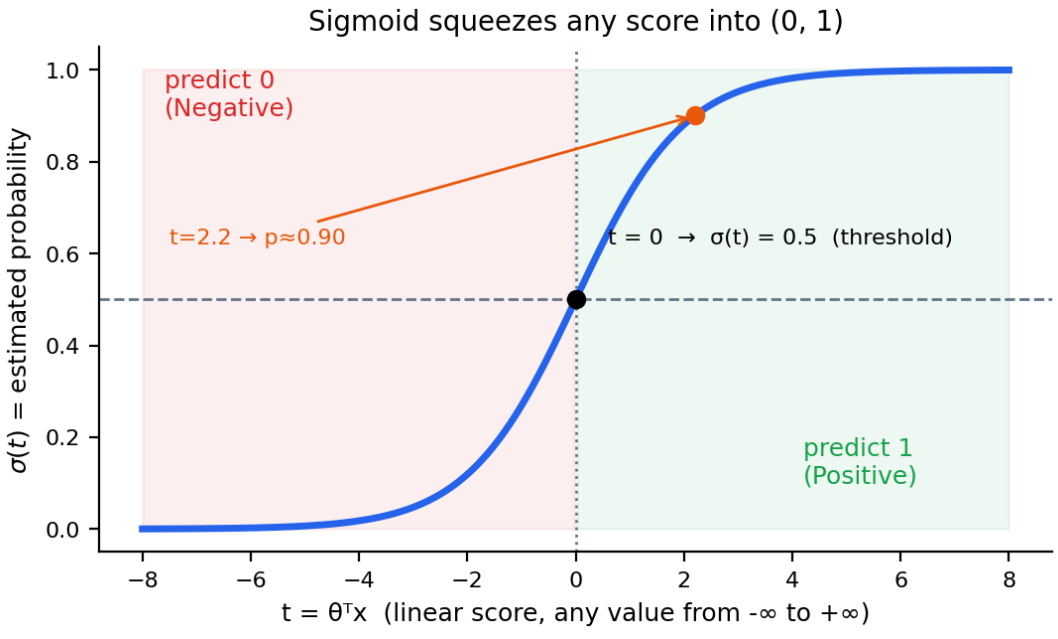
من كشكولك

ليه Sigmoid؟ لأنها تحوّل أي قيمة مهما كانت (من `−∞` إلى `+∞`) إلى قيمة بين `0` و `1`، فممكن نعتبرها Estimated Probability (مثلًا `0.9` = احتمال مرتفع إنها Positive).

Prediction: باستخدام `Threshold = 0.5`:

$$\hat{y} = 0 \text{ if } \hat{p} < 0.5, \quad 1 \text{ if } \hat{p} \geq 0.5$$

ولأن `Sigmoid(0) = 0.5` فلو `θ^T x > 0` ← `Probability > 0.5` ← Positive.



الـ Sigmoid: لما t = 0 الاحتمال بالظبط 0.5 (وده الـ threshold). t موجب = Positive، t سالب = Negative.

من كشكولك

Logit

الكتاب يسمّي الـ score قبل sigmoid اسم logit وهو مرتبط بـ log-odds، وهو inverse الـ logistic function:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = t$$

مش محتاج تغرق في الرياضيات دلوقتي، والمهم تعرف: **Probability ← Sigmoid ← linear score ← Threshold ← Prediction**.

من كشكولك

كيف يتدرّب Logistic Regression؟

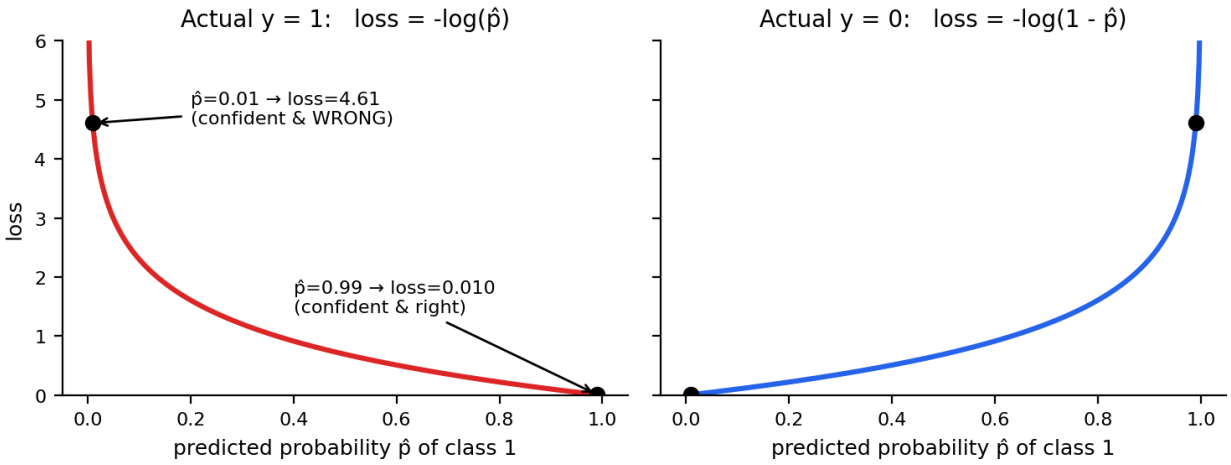
هدف التدريب: رفع الـ Probability للـ Positive instances وخفض الـ probability للـ Negative instances.

ويستخدم Log Loss / Logistic Cost Function.

لـ instance واحدة:

$$c(\theta) = -\log(\hat{p}) \text{ if } y = 1, \quad -\log(1-\hat{p}) \text{ if } y = 0$$

المعنى المهم: لو الموديل واثق جدًا في إجابة **غلط**، الـ loss تكون كبيرة جدًا. مثلًا `Actual = 1` و `Prediction probability = 0.01` ← خطأ كارثي ← loss كبيرة. لكن `Actual = 1` و `Prediction probability = 0.99` ← loss قريبة من الصفر.



من القيم الحقيقية: p̂=0.01 والإجابة الصح 1 = خسارة 4.61 (ضخمة). p̂=0.99 = خسارة 0.010 (تقريبًا صفر). المنحنى بيصعد لما الموديل بيتأكد من غلط.

من كشكولك

Log loss على الـ Dataset كلها

بناخد **متوسط** الـ cost على كل Training instances:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(\hat{p}^{(i)}\right) + (1-y^{(i)})\log\left(1-\hat{p}^{(i)}\right)\right]$$

اسمها Logistic Regression Cost Function / log loss.

إضافة

مفيش Normal Equation للـ Logistic Regression (مفيش حل مباشر مقفول)، فالتدريب بـ Gradient Descent (أو نسخ محسّنة منه). ولحسن الحظ الـ log loss هي كمان convex، فالـ GD مضمون يوصل للـ minimum العام.

مثال حقيقي على Iris: بنتنبأ هل الزهرة Iris virginica ولا لأ من عرض البتلة `petal width` بس.

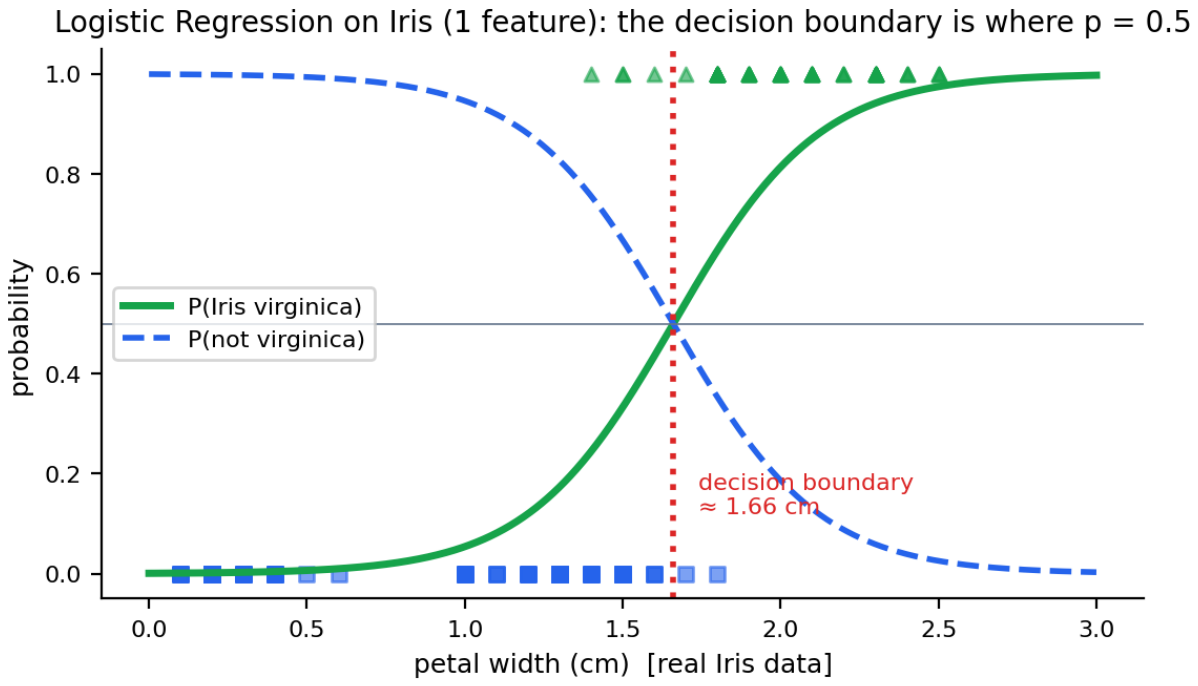
```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression

iris = load_iris(as_frame=True)
X = iris.data[["petal width (cm)"]].values
y = (iris.target == 2).astype(int)          # 1 = Iris virginica, 0 = not

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X, y)
print(log_reg.predict_proba([[1.7], [1.5]]))
print(log_reg.predict([[1.7], [1.5]]))
```

Output (real run)

```
[[0.457 0.543]
 [0.667 0.333]]
[1 0]
```



من داتا Iris الحقيقية: الاحتمال بيعدّي 50% عند عرض بتلة حوالي 1.66 سم (Decision boundary). عند 1.7 سم الموديل بيقول virginica باحتمال 54%، وعند 1.5 سم بيقول لأ باحتمال 67%.

من كشكولك

الفرق بينها وبين Linear Regression

	Linear Regression	Logistic Regression
الخطوات	Weighted sum ← Prediction Value	Weighted sum ← Sigmoid ← Probability ← Class
الناتج	رقم حقيقي	احتمال بين 0 و 1 (ثم class)
الـ Cost	MSE	Log loss

من كشكولك

Decision Boundary (الحد الفاصل)

هي زي **خط فاصل أو حائط** وهي اللي الموديل بيرسمها عشان يقرر بناءً عليها: البيانات دي تبع أي فئة (class)؟

من كشكولك

الـ C في Logistic Regression

ملاحظة مهمة من الكتاب: الـ Hyperparameter الخاص بقوة الـ Regularization في Scikit-learn `LogisticRegression` اسمه `C`، وهو **عكسي** لقوة Regularization:

- `C` ↑ ← Regularization ↓
- `C` ↓ ← Regularization ↑

إضافة

يعني عكس `alpha` في Ridge/Lasso (تقريبًا `C = 1/alpha`). والـ default هو `L2` مع `C=1.0`. لو الموديل عمل overfit قلّل `C`، ولو عمل underfit زوّد `C`.

14 الـ Softmax Regression والـ Cross Entropy

من كشكولك

- الـ Logistic Regression العادي بيستخدم لـ Binary classification؛ الفكرة لو كان عندك أكتر من class مباشرة، وده الـ Multinomial logistic Regression / Softmax Regression.
- وده مختلف عن الطريقة اللي قابلناها في Ch 3 مع: `OvR / OvO`.
- Softmax يستطيع التعامل مع Multiclass مباشرة.

من كشكولك

إزاي Softmax يشتغل لو عندنا K classes؟

أولًا: يحسب Score لكل Class:

$$s_k(\mathbf{x}) = (\theta^{(k)})^T \mathbf{x}$$

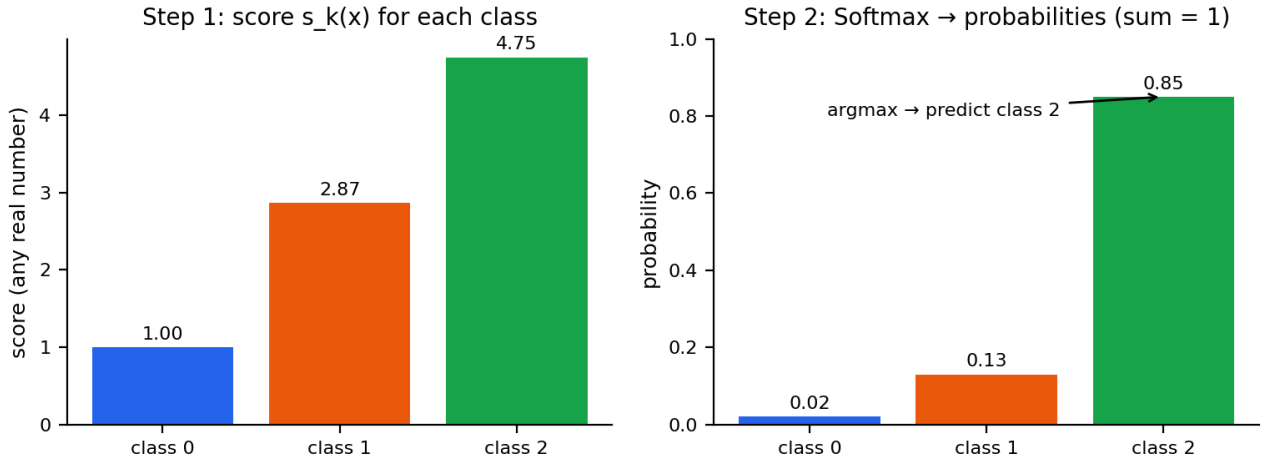
يعني كل Class لها Parameter vector خاص بها. مثلًا 3 classes: `class 0 -> score` , `class 1 -> score` , `class 2 -> score`.

من كشكولك

بعد ذلك Softmax تحوّل الـ Scores إلى Probabilities:

$$\hat{p}_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^{K} \exp(s_j(\mathbf{x}))}$$

الفكرة: يحوّل كل الـ scores إلى احتمالات **مجموعها = 1**. مثلًا: `Class 0 -> 0.02` , `Class 1 -> 0.13` , `Class 2 -> 0.85`. ∴ مجموعهم = 1.



نفس الأرقام اللي في كشكولك (0.02 / 0.13 / 0.85): شمال الـ scores الخام (أي رقم)، يمين بعد Softmax بقت احتمالات مجموعها 1. القيم حقيقية (محسوبة بالمعادلة).

من كشكولك

Prediction في Softmax

يختار صاحب أكبر Probability:

$$\hat{y} = \text{argmax}_k \, \hat{p}_k$$

أي Class صاحب أكبر Score / Probability هو الـ prediction.

من كشكولك

Softmax مع Multioutput

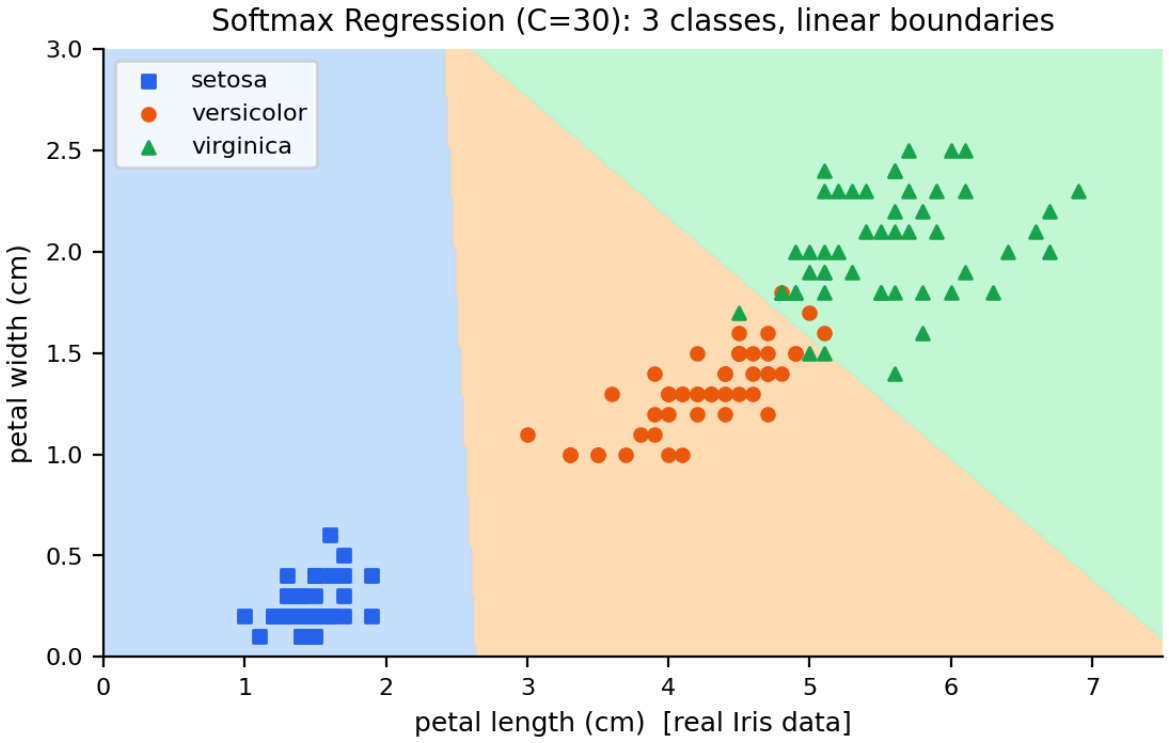
الكتاب يذكر إن Softmax هو Multiclass، يعني يختار class واحد فقط، وليس Multilabel / Multioutput.

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target                              # 3 classes: 0, 1, 2

softmax_reg = LogisticRegression(C=30, random_state=42)   # softmax by default for 3+ classes
softmax_reg.fit(X, y)
print(softmax_reg.predict([[5, 2]]))
print(softmax_reg.predict_proba([[5, 2]]).round(3))
```

Output (real run)

```
[2]
[[0.000 0.029 0.971]]
```



من داتا Iris الحقيقية: Softmax بيقسّم المستوى لـ 3 مناطق بحدود خطية. النقطة (5, 2) وقعت في منطقة virginica باحتمال 97%.

انتبه / تصحيح

في النسخ القديمة كانوا بيكتبوا `multi_class="multinomial"` عشان يفعّلوا Softmax. الـ parameter ده اتعمله deprecation في النسخ الأحدث (من 1.5 تقريبًا)، وفي النسخة اللي اتجرّب عليها الملف ده (scikit-learn 1.8) اتشال تمامًا وكتابته بتدّي `TypeError`. الـ `LogisticRegression` دلوقتي بتستخدم Softmax تلقائيًا لما يبقى فيه 3 classes أو أكتر، فلو كودك القديم فيه الـ parameter ده، شيله.

من كشكولك

Cross Entropy

كيف نُدرّب Softmax؟ نريد للموديل يعطي Probability **عالية** للـ Target class. لذلك يستخدم Cross Entropy loss:

$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$$

- في حالة class واحد صحيح: `Target: [0, 0, 1]`.
- إذا الموديل قال: `[0.01, 0.04, 0.95]` ده ممتاز.
- أما لو قال: `[0.80, 0.15, 0.05]` فـ Cross entropy تعاقبه بشدّة لأنه أعطى Probability صغيرة للـ Target class.



القيم الحقيقية من الـ loss = -log(احتمال الـ class الصح): الموديل A خسارته 0.051، والموديل B خسارته 2.996 (حوالي 58 ضعف).

إضافة

لو `K = 2` (class واحد صح وواحد غلط) الـ Cross Entropy بتتحول بالظبط لـ log loss اللي شفناها في الـ Logistic Regression. يعني الـ Log loss حالة خاصة منها.

فكرة تربط الشابتر كله: Linear ← MSE. Logistic ← Log loss. Softmax ← Cross entropy. وكلهم تدريبهم بـ Gradient Descent، وكلهم ينفع معاهم Regularization.

15 أسئلة مراجعة سريعة

جاوب الأول في دماغك، وبعدين شوف الإجابات في الصفحة اللي بعدها.

1. إيه الفرق بين الـ Parameters والـ Hyperparameters؟ ومين اللي بيحدد كل واحد؟
2. إيه هو الـ Cost Function؟ وليه الـ Training loss ممكن تختلف عن الـ Evaluation metric؟
3. اكتب الـ Normal Equation وقول معناها بكلمات بسيطة.
4. ليه `LinearRegression` في sklearn بتستخدم pseudoinverse بدل `inv` المباشرة؟
5. ليه مش بنستخدم Normal Equation دايمًا؟ وإيه اللي بيبطّئها؟
6. اشرح فكرة Gradient Descent بتشبيه الجبل والضباب، وإيه هو الـ Gradient؟
7. إيه اللي بيحصل لو الـ learning rate صغير جدًا؟ وكبير جدًا؟
8. ليه الـ Scaling مهم مع Gradient Descent؟ وهل Normal Equation محتاجة scaling؟
9. ليه الـ MSE في Linear Regression كونها convex مفيدة؟
10. إيه الفرق بين Batch و Stochastic و Mini-batch GD؟ وإيه ميزة وعيب كل واحد؟
11. يعني إيه epoch؟ وكام update في 50 epochs لو m=100 في SGD؟
12. إيه هو الـ learning schedule؟ وليه SGD محتاجه؟
13. ليه لازم نعمل shuffle للداتا مع SGD؟ (iid)
14. إزاي بنعمل Polynomial Regression بـ linear model؟ وإيه دور `include_bias=False`؟
15. إيه هو الـ Combinatorial Explosion في PolynomialFeatures؟
16. إزاي تفرّق بين Overfitting و Underfitting من الـ Learning Curves؟ وإيه الفرق في العلاج؟
17. فسّر الـ Bias والـ Variance والـ Irreducible error، وإيه هو الـ Trade-off؟
18. إيه الفرق بين Ridge و Lasso؟ وإمتى تستخدم Elastic Net؟
19. ليه لازم نعمل scaling قبل Ridge؟ وهل الـ bias θ0 بيتعمله regularization؟
20. إزاي بيشتغل الـ Early Stopping؟ وإيه الفرق بين `deepcopy` و `clone`؟
21. إزاي الـ Logistic Regression بيحوّل الـ score لـ class؟ (sigmoid + threshold)
22. ليه الـ log loss بتعاقب بشدة الموديل اللي واثق وغلط؟
23. إيه معنى `C` في LogisticRegression؟ وعلاقته بالـ regularization؟
24. إيه الفرق بين Softmax Regression و Logistic Regression؟ وإيه هو الـ Cross Entropy؟

16 إجابات أسئلة المراجعة

1. **Parameters:** الموديل بيتعلمها من الداتا (θ0, θ1, ...). **Hyperparameters:** إحنا بنحددها قبل التدريب (η, epochs, alpha, degree, C).
2. دالة بتاخد θ وترجع رقم يقيس سوء الموديل (زي MSE). الـ training loss ممكن تكون أسهل في التحسين أو فيها حدود إضافية (regularization)، لكن الـ metric النهائي ممكن يكون حاجة تانية (زي Precision/Recall).
3. `θ^= (X^T X)^-1 X^T y`. بتحسب أحسن θ (اللي تقلل MSE) مباشرة من الداتا من غير iterations.
4. لأن `X^T X` ممكن مش تبقى قابلة للعكس (features مكررة أو مرتبطة تمامًا). الـ pseudoinverse (SVD) بتشتغل حتى في الحالة دي وأكتر استقرارًا.
5. لأنها بتحتاج `(X^T X)^-1` وتكلفتها بتزيد بسرعة مع عدد الـ features n (حوالي O(n^2.4) لـ O(n^3)). لو n كبير جدًا بتبقى بطيئة.
6. إنت فوق جبل في ضباب، مش شايف الوادي بس حاسس بميل الأرض فتنزل ناحية الانحدار. الـ Gradient = مجموعة المشتقات الجزئية للـ cost بالنسبة لكل θ، بيدّي اتجاه زيادة الخطأ، فبنتحرك عكسه.
7. صغير جدًا: يوصل بس ببطء شديد. كبير جدًا: ممكن يعدّي الـ minimum ويفضل يقفز أو يبعد (diverge).
8. الـ features اللي بأحجام مختلفة بتخلّي الـ cost surface وادي طويل رفيع، فالـ GD بيزحف. الـ Normal Equation مش محتاجة scaling.
9. لأنها وعاء: فيه minimum واحد عام ومفيش local minima، فالـ GD مضمون يقرب منه (لو η مناسب وفضلنا نكمل كفاية).
10. Batch: الداتا كلها في كل step (مستقر لكن بطيء). SGD: عينة واحدة في كل step (سريع جدًا وميقدرش يستقر، مشوّش). Mini-batch: مجموعة صغيرة (توازن + بيستفيد من الـ GPU).
11. epoch = مرور واحد كامل على الـ training set. في SGD كل epoch فيها m خطوة، فـ 50 epochs × 100 = 5000 update.
12. دالة بتحدد إزاي الـ learning rate بيقل أثناء التدريب (مثلًا `t0/(t+t1)`). SGD محتاجها لأنه بيفضل يقفز حوالين الـ minimum؛ نبدأ بخطوات كبيرة وبعدين نصغّرها.
13. لو الداتا مرتبة (مثلًا كل A ثم كل B)، التحديثات بتتأثر بالترتيب ومسار التدريب بيبقى غير متوازن. الـ shuffle بيخلّي العينات تقرب من iid (مستقلة ومن نفس التوزيع).
14. بنضيف features جديدة (x², x³, ...) بـ `PolynomialFeatures` وبعدين نستخدم `LinearRegression` عادي. `include_bias=False` لأن LinearRegression بتضيف الـ intercept بنفسها.
15. عدد الـ features الجديدة بيزيد بشكل ضخم مع الـ degree وعدد الـ features الأصلية (`(n+d)!/(d!n!)`)، بسبب الأُسس والـ combinations بينهم.
16. Underfitting: المنحنيين قريبين ومرتفعين (داتا أكتر مش هتفيد؛ محتاج موديل أقوى). Overfitting: train منخفض جدًا وvalidation أعلى منه بفجوة (داتا أكتر أو regularization أو موديل أبسط بتفيد).
17. Bias: بساطة زيادة (Underfitting). Variance: حساسية زيادة للداتا (Overfitting). Irreducible: noise في الداتا نفسها. الـ Trade-off: تعقيد أعلى يقلل الـ bias ويزوّد الـ variance، فنختار نقطة التوازن.
18. Ridge (L2) بيصغّر كل الأوزان. Lasso (L1) بيصفّر بعضها (feature selection). Elastic Net مزيج منهم (r)؛ بنفضله لما الـ features كتير أو مرتبطة ببعض.

19. لأن العقوبة بتعتمد على حجم الأوزان، واختلاف الـ scales بيأثر عليها. والـ θ0 bias مش بيتعمله regularization.

20. بنوقف (أو بنحتفظ بأفضل نسخة) عند أقل validation error قبل ما يبدأ يطلع. `deepcopy` بينسخ الـ learned parameters + hyperparameters، و`clone` بينسخ الـ hyperparameters بس.

21. بيحسب t = θ^T x، بعدين sigmoid(t) يطلّع احتمال بين 0 و 1، وبعدين threshold (0.5): لو ≤ 0.5 يبقى class 1 وإلا 0.

22. لأن −log(p̂) بتروح لما لا نهاية لما p̂ يقرب من 0 والإجابة الصح 1. مثلًا p̂=0.01 تدي خسارة 4.6 مقابل 0.01 لما p̂=0.99.

23. C هو معكوس قوة الـ regularization: C كبير = regularization أضعف، C صغير = أقوى. (تقريبًا C = 1/alpha).

24. Softmax بيتعامل مع أكتر من class مباشرة (probabilities مجموعها 1 ولكل class parameter vector)، والـ Logistic العادي binary. Cross Entropy هي الـ loss بتاعته: −(1/m) Σ y log(p̂)، وبتعاقب لما الـ class الصح ياخد احتمال صغير. لو K=2 بتبقى هي الـ log loss.